Programmazione e Algoritmica

Indice

1 Concetti Iniziali **5**

2 Strutture Dati **6**

3 Linguaggi formali **7**

4 Storia degli Algoritmi **8**

5 Alfabeti **9**
5.1 Definizione di alfabeto . . . 9
5.2 Grammatica di un alfabeto . . . 9
5.3 Derivativo immediato . . . 9
5.4 Derivativo . . . 9
5.5 Linguaggio generato . . . 9
5.6 Derivazioni canoniche . . . 10
5.7 Alberi di derivazione (Parse Trees) . . . 10

6 Regole di Inferenza **11**

7 Efficienza e costo computazionale **12**
7.1 O, Ω, Θ . . . 12
7.2 Teorema Principale (Master Theorem) . . . 13
7.3 Caso 1: $f(n)$ cresce meno della ricorsione . . . 13
7.4 Caso 2: $f(n)$ cresce come la ricorsione . . . 13
7.5 Caso 3: $f(n)$ cresce più della ricorsione . . . 13
7.6 Metodo iterativo . . . 14
7.7 Alberi di ricorsione . . . 14

8 Programmazione dinamica **15**

9 Algoritmi di Ordinamento (Sort) **17**
9.1 Insertion Sort . . . 17
9.2 Selection Sort . . . 17
9.3 Bubble Sort . . . 18
9.4 Merge Sort . . . 18
9.5 Quick Sort . . . 18
9.6 Heap Sort . . . 19
9.7 Counting Sort . . . 21
9.8 Radix Sort . . . 21
9.9 Bucket Sort . . . 21

10 Linguaggio di MAO / MiniMAO **22**
10.1 Espressioni semantiche collaterali e pure . . . 22
10.2 Controllo dei tipi (Type Checking) . . . 22
10.2.1 Analisi statica . . . 22
10.3 Grammatica e linguaggio di MAO . . . 23
10.4 Funzioni in MAO . . . 24

10.5 Record di attivazione (stack frame) . . . 24
10.6 Ricorsione . . . 25

11 Strutture Dati **26**
11.1 Alberi . . . 26
11.2 Alberi binari di ricerca . . . 28
11.3 Dizionari . . . 31
11.4 Tabella Hash . . . 31

12 Grafi **34**
12.1 Grafi non orientati . . . 35
12.2 Grafi orientati . . . 35
12.3 Complessita delle operazioni (Matrice di adiacenza) . . . 35
12.4 Complessita delle operazioni (Liste di adiacenza) . . . 36
12.5 Visite sui grafi . . . 36
12.5.1 BFS . . . 36
12.5.2 DFS . . . 39
12.5.3 Ordinamento topologico e DAG . . . 41

13 Calcolabilità **42**
13.1 Esistenza dei problemi indecidibili . . . 42
13.2 Il problema dell'arresto . . . 42
13.3 Tesi di Church-Turing . . . 43
13.4 Problemi trattabili e intrattabili . . . 43

14 Complessità **44**
14.1 Differenza di potenza tra calcolatori . . . 44
14.2 Categorizzazione dei problemi . . . 44
14.3 Time e Space . . . 44
14.4 Classi di problemi . . . 44
14.4.1 SAT . . . 45
14.5 Certificati . . . 45
14.5.1 Riduzione polinomiale . . . 46
14.5.2 Teorema di Cook (1971) . . . 47

15 Altri algoritmi **48**
15.1 Array non ordinato . . . 48
15.1.1 Ricerca . . . 48
15.1.2 Inserimento . . . 48
15.1.3 Cancellazione . . . 48
15.2 Array ordinato . . . 48
15.2.1 Ricerca . . . 48
15.2.2 Inserimento . . . 48
15.2.3 Cancellazione . . . 49
15.3 Lista non ordinata (semplicemente concatenata) . . . 49
15.3.1 Ricerca . . . 49
15.3.2 Inserimento . . . 49
15.3.3 Cancellazione . . . 49
15.4 Lista ordinata (semplicemente concatenata) . . . 49

15.4.1 Ricerca . . . 49
15.4.2 Inserimento . . . 50
15.4.3 Cancellazione . . . 50
15.5 Lista doppia non ordinata . . . 50
15.5.1 Ricerca . . . 50
15.5.2 Inserimento . . . 50
15.5.3 Cancellazione . . . 50
15.6 Lista doppia ordinata . . . 51
15.6.1 Ricerca . . . 51
15.6.2 Inserimento . . . 51
15.6.3 Cancellazione . . . 51
15.7 Albero binario (generico) . . . 51
15.7.1 Ricerca . . . 51
15.7.2 Inserimento . . . 51
15.7.3 Cancellazione . . . 52
15.8 Albero binario di ricerca (BST) . . . 52
15.8.1 Ricerca . . . 52
15.8.2 Inserimento . . . 52
15.8.3 Cancellazione . . . 52
15.9 Tabella riassuntiva . . . 52

16 Cheat Sheet (MiniMAO e MAO) 53

17 Altro 61

1 Concetti Iniziali

Uno dei primi scopi dell'algoritmica è utilizzare al meglio le informazioni che si hanno.

- **Algoritmo**: sequenza di istruzioni non ambigua che, dati in input, produce un output corretto.
- **Programma**: implementazione di un algoritmo in un linguaggio preciso, non ambiguo e formalmente definito. Eseguibile da una macchina
- **Tipo**: insieme dei valori assumibili da una variabile.

Per ogni problema esistono infiniti algoritmi per risolverlo e quindi infiniti programmi.

La maggior parte degli algoritmi assume una forma astratta per adattarsi a varie situazioni che possono essere risolte con metodologie simili.

Obiettivi della programmazione:

- Efficienza
- Correttezza
- Velocità di coding (efficienza nel tempo impiegato dai developer)
- Manutenibilità

2 Strutture Dati

- **Variabile**: casella singola.
- **Array**: sequenza di caselle numerate a cui si può accedere direttamente tramite indici.

3 Linguaggi formali

Un linguaggio detto **formale** ha:

- **Sintassi**: struttura grammaticale (correttezza grammaticale).
- **Semantica**: significato (correttezza logica).

Ogni linguaggio di programmazione è un linguaggio formale.

4 Storia degli Algoritmi

Fibonacci scoprì che sui sistemi numerici posizionali possono essere applicati algoritmi per il calcolo, al contrario dei numeri romani. Il più antico linguaggio formale è il DNA.

5 Alfabeti

5.1 Definizione di alfabeto

Un **alfabeto** è un insieme finito di simboli.
Una **stringa** è una sequenza di simboli di uno specifico alfabeto (anche con ripetizioni) e ha lunghezza finita. Può anche essere vuota, con lunghezza 0, indicata con ε.

$$|baco| = 4 \quad \text{se} \quad a = \text{"baco"}$$

Un linguaggio L su un alfabeto A è sottoinsieme di A^* (tutte le possibili combinazioni dell'alfabeto).
Descrivere un linguaggio di programmazione significa descrivere quali frasi sono ammissibili.
Si usano le regole di una grammatica e un metodo riconoscitivo per la semantica.

5.2 Grammatica di un alfabeto

La grammatica è una tripla (T, N, P) dove:

- T = simboli terminali (alfabeto di riferimento)
- N = simboli non terminali
- $P \subseteq S \times S^*$ = insieme delle produzioni, con $S = T \cup N$

5.3 Derivativo immediato

Data una grammatica $G(T, N, P)$, si dice che β_1 è **derivativo immediato** di β se è possibile ottenere β_1 da β applicando una produzione.

$$\beta = \beta_1 X \beta_2, \quad X \in N$$

Esempio:

$$\beta = Exp + Exp\,Op\,2$$

Produzione: $Exp ::= Num$

$$\Rightarrow \beta_1 = Num + Exp\,Op\,2$$

5.4 Derivativo

È come il derivativo immediato, ma ottenuto tramite N sostituzioni.

5.5 Linguaggio generato

Data $G(T, N, P)$ e $X \in N$, il linguaggio di X, scritto $L(X)$, è l'insieme di tutti e soli i derivativi $\beta \in T^*$ di X.

$$L(X) = \{w \mid w \in T^*, X \Rightarrow^* w\}$$

Esempio:

$$B ::= 1 \mid 0B \mid 1B$$

Stringhe che finiscono con 1.

$$B \Rightarrow 1B \Rightarrow 10B \Rightarrow 100B \Rightarrow 1001$$

"1001" appartiene al linguaggio, "1000" no.

5.6 Derivazioni canoniche

- **Sinistra**: si sostituisce sempre il simbolo più a sinistra.
- **Destra**: si sostituisce sempre quello più a destra.

5.7 Alberi di derivazione (Parse Trees)

Rappresentano tutti i possibili percorsi per effettuare una derivazione (insiemi N e E: nodi e archi). Non possono esserci due o più archi verso lo stesso nodo e il nodo radice non ha archi entranti.

6 Regole di Inferenza

Per definire la semantica del linguaggio MAO si usano **regole di inferenza**.

$$\frac{P_1, \ldots, P_n}{Q}$$

Se tutte le premesse P_i sono vere, Q è vero.
Se non ci sono premesse, abbiamo un **assioma** (sempre vero).

Un sistema logico è un insieme di regole di inferenza per dimostrare la validità di formule.
La derivazione nel sistema logico è una sequenza di passaggi che parte dagli assiomi e giustifica una conclusione.

$$D \vdash G$$

Condizioni laterali: condizioni aggiuntive che devono essere soddisfatte oltre alle premesse.
In un linguaggio logico, ogni produzione grammaticale ha una regola di inferenza collegata.

7 Efficienza e costo computazionale

Si misura in base a:

- Costo in tempo d'esecuzione (numero di operazioni elementari, ovvero non affette dal variare dell'input)
- Costo in memoria (celle utilizzate)

Il costo (tempo/spazio) si esprime in **ordine di grandezza**, considerando solo il termine dominante.
La somma di due costi costanti, ovvero legati ad operazioni elementari, è un costo costante.

Il costo totale di un algoritmo / programma può essere rappresentato sotto forma di funzione su piano cartesiano in cui l'incognita corrisponde all'input.

La complessità è una **funzione di n**, con n come input variabile.
Per vedere se una funzione / algoritmo è più **veloce** si applicano le operazioni degli infinitesimi tipiche dell'analisi matematica.

7.1 O, Ω, Θ

O grande: si usa per indicare il limite asintotico superiore della funzione della complessità (non può mai essere maggiore).
La definizione di O grande significa che per un valore abbastanza grande di n (input) esiste un c per cui la funzione della complessità si troverà sempre al di sotto di O grande.

Ω grande: si usa per indicare il limite asintotico inferiore della funzione della complessità (non può scendere al di sotto). La definizione di Ω grande significa che per un valore abbastanza grande di n esiste un c per cui la funzione della complessità si troverà sempre al di sopra di Ω grande.

Θ grande: quando limite superiore e inferiore coincidono, si ha Θ grande. Ciò vuol dire che la funzione della complessità si trova schiacciata tra il limite superiore e inferiore (**Teorema dei carabinieri**). Attenzione, ciò non vuol dire che le c delle due condizioni siano uguali.

Con "c" intendiamo una costante moltiplicata al limite inferiore o superiore.
$\Gamma_1[\Gamma_2]$

Esempio: dimostrare:

$$3n^2 - 2n - 1 \in \Theta(n^2)$$

Bisogna trovare il valore delle c che soddisfano entrambi le condizioni:

$$C_1(n^2) \leq 3n^2 - 2n - 1 \leq C_2(n^2)$$

C1 = 2.
C2 = 3, dato che la funzione è simile, escluso il fatto che viene anche sottratto qualcosa (- 2n - 1).

7.2 Teorema Principale (Master Theorem)

Si consideri una ricorrenza della forma

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n),$$

dove $a \geq 1$ (numero chiamate ricorsive), $b > 1$ (indice di riduzione del problema) e $f(n)$ è una funzione asintoticamente positiva (complessità di ogni chiamata). Sia

$$n^{\log_b a}$$

il costo indotto dalla parte ricorsiva dell'algoritmo.
Il Teorema Principale permette di determinare l'ordine di crescita di $T(n)$ confrontando $f(n)$ con $n^{\log_b a}$. Sono possibili tre casi.

7.3 Caso 1: $f(n)$ cresce meno della ricorsione

Se esiste $\epsilon > 0$ tale che

$$f(n) = O\left(n^{\log_b a - \epsilon}\right),$$

allora

$$T(n) = \Theta\left(n^{\log_b a}\right).$$

7.4 Caso 2: $f(n)$ cresce come la ricorsione

Se esiste $k \geq 0$ tale che

$$f(n) = \Theta\left(n^{\log_b a} \log^k n\right),$$

allora

$$T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right).$$

7.5 Caso 3: $f(n)$ cresce più della ricorsione

Se esiste $\epsilon > 0$ tale che

$$f(n) = \Omega\left(n^{\log_b a + \epsilon}\right),$$

e se vale la condizione di regolarità

$$a\,f\left(\frac{n}{b}\right) \leq c\,f(n) \qquad \text{per qualche } c < 1,$$

allora

$$T(n) = \Theta(f(n)).$$

In sintesi, il Teorema Principale permette di risolvere molte ricorrenze della forma $T(n) = aT(n/b) + f(n)$ confrontando il costo ricorsivo $n^{\log_b a}$ con il costo non ricorsivo $f(n)$.

7.6 Metodo iterativo

Quando non si riesce ad applicare il teorema principale si può usare il **metodo iterativo**.
Col metodo iterativo bisogna proseguire come per induzione:

$$
\begin{aligned}
T(n) &= T(n-1) + C_1 \\
T(n) &= T(n-2) + C_1 + C_2 \\
T(n) &= T(n-3) + C_1 + C_2 + C_3 \\
&\dots
\end{aligned}
$$

Si termina con il caso base, ovvero $n - 1 = 1 = C_0$

7.7 Alberi di ricorsione

Quando non si riesce ad applicare il teorema principale si possono usare gli **alberi di ricorsione**.
Si svolge comunque la ricorsione ma si disegna una specie di albero discendente laterale, indicando ogni volta $f(n)$ per ogni step.
Il valore finale, se l'esempio è uguale a quello del metodo iterativo, sarà $f(n)\cdot$ numero di chiamate iterative.

Fooling argument: un algoritmo deve leggere tutti gli input per svolgere u bel lavoro, infatti se abbiamo n input si ha una complessita minima di n (Ωn)
Questo vale solo quando non ci sono ipotesi particolari sui dati in input, come ordinamento.

8 Programmazione dinamica

La **programmazione dinamica** (PD) è una tecnica di progettazione di algoritmi. Come il divide-et-impera, è una tecnica che si applica nei casi in cui la soluzione del problema può essere definita ricorsivamente.
A differenza del divide-et-impera però, che opera in maniera top-down fino a ridursi a sottoproblemi via via più piccoli, la PD risolve prima i sottoproblemi e poi in maniera **bottom-up** a partire dalla loro soluzione costruisce via via soluzioni a sottoproblemi più grandi fino al problema originale.

Ecco gli step da eseguire per sviluppare un algoritmo con la programmazione dinamica:

- Identificare i sottoproblemi "interessanti".
- Identificare e risolvere in modo **diretto** i sottoproblemi elementari.
- Stabilire la regola ricorsiva per il calcolo della soluzione dei sottoproblemi e dei problemi soprastanti a partire dai valori gia calcolati.
- Capire come restituire il valore del problema generale.

Vediamo un esempio con il calcolo dell'n-esimo numero di fibonacci.

$$f(0) = 0$$
$$f(1) = 1$$
$$f(n) = f(n-1) + f(n-2)$$

```
fibRec(n) { //Versione ricorsiva
    if (n = 0): return 0
    if (n = 1): return 1
    return fibRec(n-1) + fibRec(n-2)
}
```

Lo stesso problema in PD svolge i casi base e **ricorsivamente** risolve i casi superiori fino ad arrivare ad n.

```
fibPD(n) {
    f = []
    f[0] = 0
    f[1] = 1
    for (i = 2, i <= n, i++):
        f[i] = f[i-1] + f[i-2]
    return f[n]
}
```

Vediamo adesso un'altro problema: abbiamo una corda lunga n cm e una tabella di prezzi, ogni prezzo è relativo a una lunghezza specifica. Calcolare il ricavo massimo ottenibile tagliando la corda.
Nel caso avessimo una corda di 4 cm, e questi prezzi:

$$\begin{bmatrix} 1cm = 1 \\ 2cm = 5 \\ 3cm = 8 \\ 4cm = 9 \end{bmatrix}$$

La corda può essere tagliata in 2^{n-1} modi.
La **soluzione ottima** è due sotto-corde da $10cm$, vediamo come trovarlo tramite un programma:
Soluzione ricorsiva:

```
ropeCut(p,n)  {\\prezzi e lunghezza corda
    if(n=0): return 0
    q = -infty
    for (i=1,i<=n,i++):
        q = max(q,p[i] + ropeCut(p,n-1))
    return q
}
```

Soluzione PD:

```
ropeCut(p,n):
    r = []
    r[0] = 0
    for (j = 1 to n):
        q = -inf
        for (i = 1 to j):
            if (q < p[i] + r[j-1]):
                q = p[i] + r[j-1]
        r[j] = q
    return r[n]
```

9 Algoritmi di Ordinamento (Sort)

9.1 Insertion Sort

Video YouTube: *Insertion Sort Visualisation.*
L'**Insertion Sort** è un algoritmo di ordinamento di array o sequenze che funziona in questo modo:

- Parti dal secondo elemento.
- Confrontalo con quelli precedenti.
- Spostalo indietro fino a quando non incontri un elemento minore.
- Ripeti con gli elementi successivi.

Il comportamento dell'algoritmo dipende dall'input.

Caso Ottimo: L'array è già ordinato.
Complessità: $O(n)$

Caso Medio: L'array è casuale.
Complessità: $O(n^2)$

Caso Pessimo: L'array è ordinato in modo inverso.
Complessità: $O(n^2)$

```
for j = 2 to n do
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key do
        A[i + 1] = A[i]
        i = i - 1
    A[i + 1] = key
```

9.2 Selection Sort

Il **Selection Sort** ordina un array trovando di volta in volta il minimo tra gli elementi non ancora ordinati e scambiandolo con l'elemento corrente.

- Trova il minimo nell'intero array.
- Scambialo con il primo elemento.
- Ripeti sull'array rimanente.

Caso Ottimo, Medio e Pessimo: Sempre $O(n^2)$

```
for i = 0 to n - 2 do
    min = i
    for j = i + 1 to n - 1 do
        if A[j] < A[min] then
            min = j
    swap(A[i], A[min])
```

9.3 Bubble Sort

Il **Bubble Sort** confronta coppie di elementi adiacenti e li scambia se sono nell'ordine sbagliato, "spingendo" gradualmente i valori maggiori verso la fine.

Caso Ottimo: Array già ordinato, $O(n)$

Caso Medio e Pessimo: $O(n^2)$

```
for i = 0 to n - 2 do
    for j = 0 to n - i - 2 do
        if A[j] > A[j + 1] then
            swap(A[j], A[j + 1])
```

9.4 Merge Sort

Il **Merge Sort** utilizza il paradigma *divide et impera*. Divide ricorsivamente l'array in due parti, le ordina e poi le unisce.

Tutti i casi: $O(n \log n)$

```
function mergeSort(A):
    if length(A) > 1:
        mid = length(A) / 2
        L = A[0..mid-1]
        R = A[mid..end]
        mergeSort(L)
        mergeSort(R)
        merge(L, R, A)
```

9.5 Quick Sort

Il **Quick Sort** sceglie un elemento detto *pivot* e divide l'array in due sottoarray: gli elementi minori del pivot e quelli maggiori, ordinandoli ricorsivamente.

Caso Ottimo e Medio: $O(n \log n)$

Caso Pessimo: $O(n^2)$ (se il pivot è sempre il minimo o massimo)

```
function quickSort(A, low, high):
    if low < high:
        p = partition(A, low, high)
        quickSort(A, low, p - 1)
        quickSort(A, p + 1, high)
```

9.6 Heap Sort

Il **Heap Sort** costruisce un heap (albero binario quasi completo senza puntatori) e ripete: estrae il massimo (radice) e lo pone in fondo all'array. Quindi i passi sono questi:

- Crea un heap A da un array (funzione inserimento nodi)
- Scambia A[i] con A[n-1-i]
- Aggiusta l'heap non considerando la radice che è stata spostata in fondo all'array (funzione change key).
- Ripete i due passi precedenti.
- **Heap implicito:** albero completo da sinistra, viene riempito sempre a partire da sinistra.
  - **Min-heap:** il padre contiene un elemento $\leq$ di quelli dei figli.
  - **Max-heap:** il padre contiene un elemento $\geq$ di quelli dei figli.

Esiste una regola matematica implicita per usare solo l'array: si numerano i nodi con gli indici dell'array e notiamo:

$$index = 3$$
$$sx(i) = 2i + 1$$
$$dx(i) = 2i + 2$$
$$dx = sx + 1$$
$$sx = dx - 1$$
$$padre(i) = [(i - 1)/2]$$
$$\forall i$$

Bisogna, nel codice, verificare che ogni elemento i dell'array sia $\leq$ o $\geq$ (dipende dall'heap) dei suoi figli (indici dei figli indicati sopra).
Più semplicemente basterebbe controllare l'opposto, la condizione dei figli rispetto al padre ad esclusione dell'indice 0 (root).
Per l'inserimento di nuovi nodi, se il valore è già piu piccolo del valore del padre a cui verrà assegnato allora si aggiunge tranquillamente, altrimenti si scambiano il nuovo valore e il padre, continuando cosi fino a che è giusto.
Questa operazione si chiama **Heapify**.

Altezza di un heap implicito: $\log n$ con n =nodi dell'heap.
$A[n-1]$ ha sicuramente distanza dalla radice = altezza heap.

Eliminazione della radice: si elimina la radice dall'array, il buco viene riempito dall'ultimo elemento spostandolo in cima al posto della radice, si scambia la nuova radice con un figlio fino a che non si riforma un heap.

Aggiornamento del valore di un nodo: sostituiamo inizialmente il valore di un nodo a x, se x è già maggiore dei figli e minore del padre basta sostituire il valore, altrimenti:

- **X < almeno 1 figlio:** si scambia il nodo con valore x con quello del figlio maggiore di x.
- **X > almeno 1 figlio:** si scambia il nodo con valore x con il padre.

Tutti i casi: $O(n \log n)$

```
isMaxHeap(A):
    for i=1 to n do:
      if (A[i]>A[parent(i)]) return false
    return true

maxHeapify(i):
    if(i==0) return //siamo alla radice
    if(A[i] > A[parent(i)]):
        swap(A[i],A[parent(i)])
        maxHeapify(parent(i))

buildHeap(A):
    for i=1 to |A|-1
        maxHeapify(i)

heapChangeKey(A,i,k):
    A[i] = k
    temp1 = 0
    temp2 = 0
    while(temp1 = temp2):
        temp1 = i
        if (left(i) < A.size && A[i] < A[2*i + 1]):
            swap(A[i], A[2*i+1])
            i = 2*i + 1
        if(right(i) < A.size && A[i] < A[2*i + 2]):
            swap(A[i], A[2*i + 2])
            i = 2*i + 2
        if(i > 0 && A[i] > A[parteIntera((i-1)/2)]):
            swap(A[i], A[parteIntera((i-1)/2)])
            i = parteIntera((i-1)/2)
        temp2=i

heapSort(Arr):
```

9.7 Counting Sort

Il **Counting Sort** non confronta elementi ma conta le occorrenze di ciascun valore, adatto a numeri interi di piccolo intervallo.

Tutti i casi: $O(n + k)$ dove k è il range dei valori.

```
countingSort(A, k):
    C = array[0..k] of 0
    for i = 1 to n:
        C[A[i]] = C[A[i]] + 1
    for i = 1 to k:
        C[i] = C[i] + C[i - 1]
    for j = n downto 1:
        B[C[A[j]]] = A[j]
        C[A[j]] = C[A[j]] - 1
```

9.8 Radix Sort

Il **Radix Sort** ordina numeri digit per digit (es. per cifra decimale), partendo dalla cifra meno significativa, usando un algoritmo stabile come il Counting Sort.

Tutti i casi: $O(d(n + k))$, dove d è il numero di cifre.

```
radixSort(A, d):
    for i = 1 to d:
        countingSortByDigit(A, i)
```

9.9 Bucket Sort

Il **Bucket Sort** divide gli elementi in "secchi" (bucket) in base al valore, li ordina singolarmente (es. con Insertion Sort) e poi li concatena.

Caso Ottimo e Medio: $O(n + k)$

Caso Pessimo: $O(n^2)$ (tutti gli elementi nello stesso bucket)

```
bucketSort(A):
    n = length(A)
    create n empty buckets B[0..n-1]
    for i = 0 to n - 1:
        index = floor(n * A[i])
        insert A[i] into B[index]
    for i = 0 to n - 1:
        insertionSort(B[i])
    concatenate(B[0..n-1])
```

10 Linguaggio di MAO / MiniMAO

10.1 Espressioni semantiche collaterali e pure

Espressione con effetto collaterale: cambia lo stato.

$$\langle E, \rho, \sigma \rangle \Downarrow \langle \rho_1, \sigma_1 \rangle$$

Espressione pura: mantiene lo stato e restituisce un valore.

$$\langle E, \rho, \sigma \rangle \Downarrow V$$

10.2 Controllo dei tipi (Type Checking)

Nonostante aver definito una grammatica per MiniMAO, ci sono comandi che sono sintatticamente corretti ma che potrebbero essere scartati a priori.
Per esempio:

$$\text{int x} = \mathit{true} + 3 \rightarrow \text{ sintatticamente giusto, ma non ha senso e da errore.}$$

Ecco alcuni tipi di errori:

- Variabile utilizzata prima della sua dichiarazione.
- Variabile non inizializzata (Ogni variabile quando viene dichiarata deve assumere un valore in Mao).
- Operazioni non definite, come dividere per 0 o accedere a degli indici inesistenti.

10.2.1 Analisi statica

L'analisi statica di un programma viene effettuata al momento di generazione / compilazione e rileva possibili errori, evidenziandoli nell'IDE.
Lo svolgimento dell'analisi NON esegue il programma.
Questa analisi si limita al controllo dei tipi (**type checking**), che permette di rilevare molti problemi legati alla mancanza di coerenza tra tipi delle variabili.
Diversi linguaggi attuano questa analisi in modo diverso.

Sistema di tipi: in Mao, definiamo il controllo dei tipi in modo formale attraverso un sistema logico mediante le cosidette **regole di tipo** che stabiliscono le condizioni necessarie affinchè un'espressione o un comando siano considerati ben tipati.
Il controllo è fatto in maniera **composizionale**, ogni espressione e comando deve forzare dei tipi in ogni loro elemento di composizione.

Ambiente di tipo: funziona parziale $\Gamma : I \rightarrow T$ che assegna un tipo ad un espressione o comando.

Variabili libere: identificatori ancora non definiti.
Si definisce la funzione **free variables**: $fv : E \to I$.

- $fv(V) = \emptyset$
- $fv(Id) = Id$
- $fv(E_1 \text{ bop } E_2) = fv(E_1) \cup fv(E_2)$
- $\dots$

Per esempio:

$$fv(x + 3) = fv(x) \cup fv(3) = fv(x) = x$$

La funzione può lavorare anche su C: $fv : C \to I$.

- $fv(C_1C_2) = fv(C_1) \cup (fv(C_2) \setminus dv(C_1))$
- $fv(TId = E) = fv(E)$

Variabili dichiarate: identificatori gia dichiarati pronti all'uso.
Si definisce la funzione: $dv : C \to I$.

Giudizio di tipo per le espressioni: dato un ambiente Γ e un'espressione E:

$$\Gamma \vdash E : T$$

Si legge che E, nell'ambiente Γ, è ben tipata.
Esempio:

$$\Gamma = x : int \vdash x + 3 : int$$

Giudizio di tipo per comandi: dato un ambiente Γ e un comando C:

$$\Gamma \vdash C : \Gamma^1$$

Si legge che C, in un ambiente Γ, rende disponibile l'ambiente Γ^1

10.3 Grammatica e linguaggio di MAO

In MAO il linguaggio viene esteso includendo i tipi:

- **char:** per rappresentare simboli, lettere e altri caratteri alfanumerici, esistono caratteri speciali rappresentati da una **sequenza di escape**, come '\ n' e '\ t'.
- **string:** rappresentate come array di char.

Nel linguaggio di MAO viene introdotto anche l'assegnamento multiplo, ecco un esempio:

$$letx, y, z = 3, 4, 5;$$

Un altro strumento introdotto in MAO è la direttiva **return**, assieme alle **funzioni**.

10.4 Funzioni in MAO

Principio di astrazione: le funzioni assumono una forma astratta in modo da risolvere molteplici problemi con lo stesso codice.
Le funzioni servono quindi per ridurre al minimo le duplicazioni di codice nel programma.

Principio di corrispondenza: parti di un programma che hanno effetti simili devono avere una sintassi simile. Facilita l'apprendimento e l'interpretazione del programma.

Ogni funzione ha:

- Nome
- Definizione con parametri in input (**formali**)
- Tipo di return
- Corpo di definizione
- Eventuali chiamate (con parametri **attuali**)

Le variabili libere di una funzione sono quelle che non hanno una definizione ne' dentro ne' fuori al corpo della funzione stessa. Per evitare errori, se non dichiarata all'interno del corpo ma utilizzata comunque, una variabile deve essere dichiarata all'esterno del corpo, per esempio globalmente.

Scoping statico: le locazioni (non i valori) delle variabili libere delle variabili e le variabili gia definite fuori dal corpo sono salvate al momento della dichiarazione della funzione.
Questo viene garantito costruendo delle **chiusure**.

Scoping dinamico: non si può applicare lo scoping statico.

Dopo aver effettuato una chiamata con un parametro attuale, ecco cosa succede:

$$\text{formalParameter} = \text{actualParameter}$$
$$\forall parameters$$

10.5 Record di attivazione (stack frame)

Contiene le informazioni necessarie all'esecuzione del blocco della funzione, ha due puntatori:

- **Catena dinamica:** rappresenta la sequenza di chiamate e serve a garantire il corretto ordine di esecuzione, stato del Program Counter.
- **Catena statica:** implementa scoping statico e garantisce che i nomi siano referenziati rispettando la visibilità di variabili e funzioni.

Altre cose che contiene:

- **Indirizzo di ritorno:** indirizzo dell'istruzione da eseguire al termine della funzione / blocco corrente.

- **Indirizzo risultato:** indirizzo all'interno del record di attivazione del chiamante dove si va a scrivere il risultato.
- **Parametri:** spazio riservato alla associazione parametri formali - parametri attuali.
- **Variabili locali:** spazio riservato alla allocazione delle variabili locali al blocco
- **Risultati temporanei:** spazio riservato alla allocazione delle variabili temporanee generate dal compilatore.

Ogni volta che si chiama una funzione si crea un nuovo record di attivazione.
Questo implica che usare funzioni costa di più che usare un semplice pezzo di codice, a meno che non ci sia duplicazione di codice.
I record di attivazione sono organizzati come una **pila**, ovvero seguono la politica **LIFO** (Last In First Out).
Per l'appunto, gli stack frame compongono lo **stack**. Se chiamiamo troppe funzioni (anche tramite ricorsione infinita) si va incontro ad uno **stack overflow**.
L'indirizzo del top dello stack si chiama **Stack Pointer**.

Heap: parte di memoria in cui si memorizzano dati dinamici, come liste. Si tratta di una memoria frammentata, se richiediamo troppe risorse si ottiene **heap overflow**.

10.6 Ricorsione

Una funzione ricorsiva si chiama da sola, esistono chiamate annidate della stessa funzione.
In una funzione ricorsiva è ammessa una variabile libera a patto che corrisponda al nome della funzione stessa.
Molto utile per strutture di dati e algoritmi.

Ricorsione in coda: la chiamata ricorsiva è l'ultimo comando della funzione e quindi ogni stack frame varia solo nei parametri, per questo si evita di duplicarli.

Mutua ricorsione: una funziona è definita in termini dell'altra e viceversa.

Ricorsione diretta: la chiamata ricorsiva si trova nel blocco della funzione stessa.

Ricorsione indiretta: mutua ricorsione.

11 Strutture Dati

11.1 Alberi

Albero binario: albero in cui ogni nodo ha al massimo 2 figli, uno sinistro e uno destro.

Simbologia:

- u = nodo
- $u.key$ = valore nodo
- $u.right$ = figlio destro (nodo)
- $u.left$ = figlio sinistro (nodo)

Una **visita** è un algoritmo che lavora su tutti i nodi in sequenza, se ogni nodo impiega un tempo costante, la complessità della visita è n.

Tipi di visite:

- Visita anticipata = si lavora prima sulla radice.
- Visita posticipata = prima si lavora sulla parte sinistra, poi la destra e poi la radice.
- Visita simmetrica = si lavora prima sulla parte sinistra, poi sulla radice e poi sulla parte destra.

ABCB = Albero binario completamente bilanciato: completo e tutti i nodi hanno lo stesso albero a destra e sinistra (forma). I nodi di un albero binario completo o hanno due figli o sono foglie.
Un ABCB ha numero massimo di foglie = 2^h con h =altezza dell'albero, inoltre ha $2^h - 1$ nodi interni e $2^{h+1} - 1$ nodi totali.
Al contrario, l'altezza $h = log(n + 1) - 1$.
Un albero binario è 1-bilanciato se e solo se per ogni nodo il valore assoluto della differenza delle altezze dei suoi sottoalberi è ≤ 1.

Nodi cardine: nodo di cui la profondità corrisponde all'altezza.

Nodi centrali: nodo la cui sommatoria con i nodi soprastanti corrisponde al numero di nodi del sottoalbero radicato in se stesso.

```
raddoppiaPari(u): //Raddoppia il valore di ogni nodo
  if (u != nul):
      u.key = u.key * 2
      raddoppiaPari(u.left)
      raddoppiaPari(u.right)

visitaSimmetrica(u):
  if (u != nul):
      visitaSimmetrica(u.left)
```

```
        analisiNodo(u)
        visitaSimmetrica(u.right)

dimensione(u):
  if (u == nul):
        return 0
  else:
        return 1 + dimensione(u.left) + dimensione(u.right)

altezza(u):
  if (u == nul):
        return -1
  else:
        hs = altezza(u.left)
        hd = altezza(u.right)
        return 1 + max(hs,hd)

\\Complessita costante se decomp e ricomb hanno complessita costante.
\\deconmponibile: valore da calcolare per sottoalbero radicato in u
decomp(u):
  if (u == nul):
        return decomponibile($\lambda$)
  else:
        risSx = decomp(u.left)
        risDx = decomp(u.right)
        return ricombina(risSx,risDx)

nodiCardine(u,p): \\p = profondita, 0 nella radice
     if (u == nul):
          return
     if (p == altezza(u)):
          console.log(u)
     nodiCardine(u.left,p+1)
     nodiCardine(u.right,p+1)

nodiCentrali(u,sum): \\sum = sommatoria nodi u e soprastanti
     if (u == nul):
          return 0
     else:
          dimS = nodiCentrali(u.left, sum+u.key)
          dimD = nodiCentrali(u.right, sum + key)
          dim = dimS + dimD + 1
          if (dim == sum + u.key):
               printf u.key
          return dim

contaFoglie(u): \\u = un albero binario
     if (u == nul):
```

```
        return 0
    if (u.left == nul && u.right == nul):
        return 1
    else:
        sx = contaFoglie(u.left)
        dx = contaFoglie(u.right)
        return sx + dx

isCompleto(u):
    if(u == nul):
        return true
    if(u.left == nul && u.right == nul):
        return true
    if(u.left != nul && u.right != nul):
        return isCompleto(u.left) && isCompleto(u.right)
    else:
        return false

\\restituisci quante foglie hanno la chiave doppia di quella del padre.
foglieDoppio(u,fatherKey):
    if(u == nul):
        return 0
    if(u.left == nul && u.right == nul && (u.key / 2) == fatherKey):
        return 1
    else:
        return foglieDoppio(u.left,u.key) + foglieDoppio(u.right,u.key)

is1Bilanciato(u):
    if(u == nul):
        return true
    return (is1Bilanciato(u.left)
    && is1Bilanciato(u.right)
    && Math.abs(altezza(u.left) - altezza(u.right)) <= 1)

\\output: true se esistono due elementi che sono uno l'opposto dell'altro
esistonoOpposti(a): \\array non ordinato a
\\caso pessimo: scorrere tutto o ordinarlo e poi fare i check. Complessita
\\caso medio: si usa una tabella hash, si riempie la tabella e poi si fa il
```

11.2 Alberi binari di ricerca

Un **albero binario di ricerca** è un insieme dinamico su un albero binario.
Per ogni nodo di valore k dell'albero binario:

- **Sottoalbero sinistro:** $< k$
- **Sottoalbero destro:** $> k$

Ogni sottoalbero di un ABR è un ABR.
In un ABR, la visita simmetrica ritorna sempre una sequenza crescente.
Per eliminare un nodo si fa cosi:

- Se è una foglia si elimina e basta
- Se ha un solo figlio lo si elimina e si sostituisce con il suo sottoalbero
- Se ha due figli si trova il successore y del nodo che ha al massimo un figlio e poi si mette y al posto di z ed elimino il vecchio y dall'albero.

Delete in ABR: se è una foglia, il nodo si toglie, altrimenti si elimina e si sostituisce con il suo unico figlio, se ne ha due di figlia allora (cerca nelle slide).

```
search(x,k): //Complessita O(h)
    if((x == nul) || (x.key == k)):
        return x
    if(x.key > k):
        return search(x.left,k)
    if(x.key < k):
        return search(x.right,k)

abrMin(x):
    if(x == nul):
        return nul
    while(x.left != nul) do:
        x = x.left
    return x

abrMax(x):
    if(x == nul):
        return nul
    while(x.right != nul) do:
        x = x.right
    return x

successore(x): //Complessita O(n)
    if(x == nul):
        return nul
    if(x.right != nul)
        return abrMin(x.right)
    y = x.p //padre
    while((y != nul) && (x == y.right)) do:
        x = y
        y = y.p
    return y

insert(u,z): //u = abr
    y = nul //inseguitore
```

```
    x = u
    while(x |= nul) do:
        y = x
        if(z.key < x.key):
            x = x.left
        else
            x = x.right
    //Usciti dal while x = nul e y = il parent
    //x va attaccato al parent
    z.parent = y
    if(y = nul):
        u = z
    else:
        if(z.key < y.key):
            y.left = z.key
        else:
            y.right = z.key

delete(u,z):
    if((z.left == nul) && (z.right == nul)):
        y = z
    else:
        y = successore(z)
    if(z.left != nul):
        x = y.left
    else:
        x = y.right
    if(u != nul):
        u.parent = y.parent \\cancello y
    if(y.parent == nul): //y radice
        u = x
    else:
        if(y == (y.parent).left):
            (y.parent).left = x
        else:
            (y.parent).right = x
    if(y != z):
        z.key = y.key
    return y

isABR(u) {
    if(u == nul) return true
    if(!isABR(u.left)) return false
    if(u.key < prec) return false
    prec = u.key
    return isABR(u.right)
}
```

```
swap(u) {//ABR dove sx = maggiore e viceversa,scambia sottoalberi
        swap(u.left)
        swap(u.right)
        temp = u.left
        u.left = u.right
        u.right = t
}

arrayToABRBilanciato(arr,p,r) { //array ordinato
    if(p > r): return nul
    q = (p+r)/2
    u = newNode()
    u.key = A[q]\\meta
    u.left = arrayToABRBilanciato(A,p,q-1)
    u.right = arrayToABRBilanciato(A,q+1,r)
    return u
}
```

11.3 Dizionari

Un **dizionario** è una struttura dati che memorizza un insieme dinamico di elementi e supporta le operazioni di:

- Inserimento
  - $Insert(S, v)$ con $s = dizionario$, $v = value$.
- Cancellazione / rimozione
  - $Delete(S, k)$ con $s = dizionario$, $k = key$, può rimuovere anche in base a valore.
- Ricerca / verifica di esistenza degli elementi.
  - $Search(s, k)$ con $s = dizionario$, $k = key$.
  - $Locate(s, v)$ con $s = dizionario$, $v = value$.

Se in un dizionario si può solo cercare elementi e verificare la loro esistenza allora si indica come **dizionario statico**, altrimenti è un **dizionario dinamico**.
Ovviamente ci possono essere anche altri query, come $Min(s)$, $Max(s)$ ...

11.4 Tabella Hash

Le tabelle hash al caso pessimo si comporta come un array non ordinato riguardo le complessita dei metodi:

- Search
- Insert
- Delete

Nel caso medio invece ha complessita lineare.
Se la capacità della tabella hash è più piccola dei dati da rappresentare si utilizza il metodo hash che riduce notevolmente gli spazi necessari da usare per la memorizzazione, vediamo un esempio:

$$S = 1032, 787, 424, 18, 636, 321$$

$$hash(x\%m), m = 11$$

$$hash(S) = 9, 6, 9, 7, 9, 2 = \text{indici nella tabella in cui vengono salvati i numeri di S}$$

Come possiamo vedere ci sono delle **collisioni**, problema ricorrente.
Il costo computazionale del tempo è n/m e si chiama fattore di carico (n è il numero degli elementi di S), corrisponde alla lungheza media dell'array $T[i]$ (indice).
Per risolvere le collisioni si usano le **liste di trabocco** (cerca info) oppure:

- **Tabelle Hash di concatenamento:** ogni posizione ha un puntatore ad una lista di valori.
- **Tabelle Hash con indirizzamento aperto (Open Hash Tables:)** se un indice è vuoto allora si assegna normalemnte il valore, altrimenti cerco una posizione alternativa seguendo un ordine per esaminare le altre celle.
  Anche l'ordine, oltre all'indice, dipende dal valore da immettere. Tale ordine si chiama **sequenza di ispezione**.
  In generale, la sequenza di ispezione è diversa per ogni valore.
  L'hash in una tabella del genere ha un parametro extra oltre al valore, ovvero il numero del tentativo corrente (ordine di ispezione):

  $$h : U(valori)X[0, m-1] \to [0, m-1]$$

  L'ordine di ispezione è minore della grandezza della tabella.

  $$h(k, 0) \to h(k, 1) \to h(k, 2) \to \cdots$$

Esercizio di esempio di Hash Table aperte:

$$m = 11$$

$$S = \{10, 22, 31, 4, 15, 28, 17, 88, 59, 26\}$$

$$h(k, i) = (k \mod 11 + i) \mod 11$$

Doppio Hash: uno dei metodi migliori e con le migliori prestazioni, ho due funzioni hash ausiliarie h_1 e h_2.

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) mod m$$

```
\\Ottimo: lineare, pessimo: n == m
insert(T,k): \\senza cancellazioni
    i = 0
    repeat:
        j = h(k,i)
        if(T[j] == nul):
```

Chiavi	sequenza di ispezione
10	10
22	0
31	9
4	4
15	4 (collisione) ⇨ 4+1 = 5 è libera? Si ⇨ 5
28	6
17	6 (collisione)
88	0 (collisione) etc...
59	4 (collisione)
26	4 (collisione)

0	22
1	88
2	
3	
4	4
5	15
6	28
7	17
8	59
9	31
10	10

```
            T[j] = k
            return k
        else:
            i++
    until (i == m)
    throw Error(overflow tabella)

\\Ottimo: lineare, pessimo: n == m
search(T,k):
    i=0
    repeat:
        j = h(k,i)
        if(T[j] == k):
            return j
        i++
    until (T[j] == nul || i >= m)
    return nil
```

12 Grafi

I grafi sono un tipo di struttura dati e sono rappresentati da una coppia:

$$G = (V, E)$$

$$V = nodi/vertici$$
$$E = archi(u, v) : u, v \in V$$

Esistono grafi:

- **Orientati:** composti da coppie ordinate, un arco ha una direzione specifica, ha al massimo n^2 archi.
$$(u, v) \neq (v, u)$$
- **Non orientati:** un arco è bidirezionale, ha al massimo $\frac{n(n+1)}{2}$ archi.
$$(u, v) \equiv (v, u)$$

L'**ordine** di G è il numero di vertici, ovvero $|V|$.
La **dimensione** di un grafo è:

$$dim(G) = |V| + |E|$$

Se $(u, v) \in E$ allora:

- u e v sono **adiacenti** (o vicini).
- (u, v) è **incidente** su u e v.
- $\forall v$, il **grado** di v, indicato con $\delta(v)$, corrisponde al numero di archi incidenti su v.
- v è **isolato** $\Leftrightarrow \delta(v) = 0$.
- $u \rightsquigarrow v$ è il **cammino** tra tali nodi ed è una sequenza di nodi a due a due adiacenti che parte da u e finisce in v.
- Un cammino è un **ciclo** se torna al nodo di partenza.
- Un cammino è **semplice** se ha tutti nodi distinti e quindi non contiene cicli.
- $\delta(u, v)$ è la **distanza** tra due nodi, che corrisponde alla lunghezza del cammino minimo tra i due.

Un grafo può essere anche:

- **Sparso:**
$$|E| = O(n)$$
(al massimo n)
- **Denso:**
$$|E| = \Theta(n^2)$$

- **Completo (o Clique / Cricca)**: ogni coppia di vertici distinti è adiacente, massimo numero di archi possibile.
- **Connesso:** se ogni coppia di nodi è connessa, ovvero esiste un cammino che li connette.
- **Aciclico(DAG):** è senza cicli.

Un grafo $G^| = (V^|, E^|)$ è **sottografo** di $G = (V, E)$ se:

$$V^| \subseteq V$$

$$E^| \subseteq E$$

Una **componente connessa** di un grafo è un sottografo massimale di tale grafo.

I grafi possono essere rappresentati come:

- **Matrici di adiacenza:** le righe e le colonne rappresentano i nodi, le celle indicano se esiste l'arco tra quella coppia di nodi o meno e occasionalmente anche un **valore** se il grafo è **pesato**.
- **Liste di adiacenza:** rappresentato da una lista di liste, ogni indice della lista principale contiene una lista dei nodi a cui il nodo contenente la lista è collegato. La lista è *nul* se il grado del nodo è 0.

12.1 Grafi non orientati

Nei grafi non orientati vale questo lemma:

$$\sum_{v \in V} \delta(v) = 2|E|$$

12.2 Grafi orientati

Nei grafi orientati vale questo lemma:

$$\sum_{v \in V} \delta(v) = |E|$$

In questo tipo di grafo, i nodi hanno:

- **Grado in entrata:** $\delta^+(u)$
- **Grado in uscita:** $\delta^-(u)$

Il grado di un nodo corrisponde alla somma di questi due.
Un grafo ordinato è **fortemente connesso** se ogni coppia di nodi è connessa.
La definizione è analoga per **componente fortemente connessa**.

12.3 Complessita delle operazioni (Matrice di adiacenza)

Operazione	Grafo orientato	Grafo non orientato	Note
adiacenti(u,v)	$\Theta(1)$	$\Theta(1)$	return[u,v]
grado(n)	$\Theta(n)$	$\Theta(n)$	scansiono la riga u di M
aggiungiArco(u,v)	$\Theta(1)$	$\Theta(1)$	M[u,v] = 1 (M[v,u] = 1)
rimuoviArco(u,v)	$\Theta(1)$	$\Theta(1)$	analogo con "0"

12.4 Complessita delle operazioni (Liste di adiacenza)

Operazione	Grafo orientato	Grafo non orientato	Note
adiacenti(u,v)	$\Theta(\delta(u))$	$\Theta(\delta^-(u))$	find$(adj(u), v)$
grado(n)	$\Theta(\delta(u))$	$\Theta(\delta^-(u))$	adj(u).size
aggiungiArco(u,v)	$\Theta(1)$	$\Theta(1)$	$\Theta(\delta(u))$ se liste ordinate
rimuoviArco(u,v)	$\Theta(\delta(u))$	$\Theta(\delta(u) + \delta(v))$	

Ovviamente per aggiungiArco con liste ordinate e grafo non orientato la complessità è $\Theta(\delta(u) + \delta(v))$.

12.5 Visite sui grafi

Una visita su un grafo è un esame sistematico di tutti i vertici e di tutti gli archi di un grafo. (Oppure limitatamente di vertici e archi raggiungibili da una sorgente).

12.5.1 BFS

La visita **BFS (Breadth First Search)**, ovvero la **visita in ampiezza**, se $s \in V$ è nodo sorgente allora:

$$bfs(G, s)$$

scopre tutti i vertici raggiungibili da s in ordine crescente rispetto alla distanza da s. Questo algoritmo utilizza una coda e visita tutto il grafo solo se è connesso.
Vediamo un esempio, diciamo che:

- $v.d$ = distanza da s di v.
- $v.status$ = può essere "A" (da visitare), "B" (Visitato), "C" (Visitato e visitati tutti gli archi uscenti).
- $v.\pi$ = predecessore (vertice da cui si arriva in v)

```
forEach v do:
    v.d = +inf
    v.pi = nul
    v.status = A
s.status = B
s.d = 0
s.pi = nul
Q = nuovaCoda()
Enqueue(Q, s)
while(Q != empty) do:
    u = dequeue(Q)
    forEach v in adj[u] do:
        if(v.status == A):
            v.status = B
            v.pi = u
            v.d = u.d + 1
        Enqueue(Q,v)
    u.status = C
```

Spiegazione del codice: si inizializza tutto come si deve, il nodo sorgente ovviamente non ha predecessori ed è il primo da visitare (primo nella coda).
Dopo, si visita ogni nodo adiacente del primo nodo e li mette in coda (non è specificato nessun ordine specifico). Una volta visitati tutti i vicini, il nodo iniziale viene segnato col grado C e viene tolto dalla coda.
Il nodo in testa adesso è il secondo della visita e si deve ripetere il procedimento.

Complessità: $\Theta(|V| + |E|) = \Theta(n + m)$
Vediamo adesso un metodo che stampa un percorso minimo dopo aver svolto una BFS:

```
print-path(G,s,v):
    if(v == s):
        print "s"
        return
    if(v.pi == nul):
        "v non e raggiungibile"
    else:
        print-path(G,s,v.pi)
        print "v"
```

Complessità: $\Theta(1)$ se il path non esiste, altrimenti $\Theta(d(s, v))$, ovvero la distanza tra i due nodi.

Dimostriamo adesso una proprietà della visita BFS tramite alcuni lemmi.

Proprietà: la BFS calcola la distanza $\delta(s, v)$ di ogni nodo dalla sorgente e dopo l'esecuzione tale valore è salvato in $v.d$.

Lemma 1

$$G = (V, E), s \in V$$

$$\forall (u, v) \in E, d(s, v) \leq d(s, u) + 1$$

Un cammino da s a v può esistere più breve di quello passando da u, ma se ne esistesse uno più lungo non verrebbe attribuito a $v.d$.

Lemma 2: al termine di un BFS(G,s):

$$\forall v \in V, v, d \geq d(s, v)$$

Dimostrazione Lemma 2: vediamo prima il caso base:

$$s.d = 0 = \delta(s, s)$$

$$\forall v \in V \neq s, v.d = +\infty \geq d(s, v)$$

Quindi dopo il primo Enqueue, tutti i nodi tranne s hanno sempre la distanza settata a infinito.

Vediamo adesso il caso induttivo con l'ipotesi induttiva: $u.d \geq \delta(s, u)$ e sapendo che il nodo v viene scoperto da u.

$$v.d = u.d + 1 \geq \delta(s, u) + 1$$

Questa sopra è l'ipotesi induttiva applicata a cosa avviene nel ramo "then".

$$\delta(s, u) + 1 \geq \delta(s, v)$$

Questo sopra è il Lemma 1, dimostrazione completata dato che questo valore non verrà più modificato.

Lemma 3: dopo l'esecuzione di un BFS, se:

$$Q = \{v_1, v_2, \ldots, v_r\}$$

Allora:

$$v_r.d \leq v_1.d + 1$$
$$v_i.d \leq v_{i+1}.d$$
$$\forall i = 1, \ldots, r - 1$$

Ricordiamo che con v_r si intende l'ultimo elemento della coda in uno specifico momento, non l'ultimo nodo in assoluto da visitare.
Tutto questo si può dimostrare dal concetto di BFS, ovvero il fatto che tale algoritmo procede a "livelli" e una volta superato un livello ne dimentica un altro.

Corollario 4: anche questo serve a dimostrare la proprietà. I valori delle distanze da s dei vertici inseriti nella Queue sono monotoni crescenti (debolmente crescenti o strettamente crescenti) e quindi:

$$v_i.d \leq v_j.d$$

Se v_i viene inserito prima di v_j nella Queue.

Teorema di correttezza di BFS: ricapitolando, un BFS scopre tutti i vertici che sono raggiungibili da s sorgente e alla fine della sua esecuzione si avverano queste cose:

1. $\forall v \in V, v.d = \delta(s, v)$.
2. $\forall v \neq s$, con v raggiungibile da s, uno dei cammini minimi da s a v è un cammino minimo da s a $v.\pi$ seguito da $(v.\pi, v)$.

Dimostrazione per assurdo punto 1: supponiamo che:

$$\exists v : v.d \neq \delta(s, v)$$

Dato il Lemma 2 però, viene implicato questo:

$$v.d > \delta(s, v)$$

Consideriamo anche che $\delta(s, v)$ sia la distanza minima tra quelle dei vertici che soddisfano la condizione assurda e che un nodo u precede v in un cammino minimo da s a v e quindi:

$$\delta(s, v) = \delta(s, u) + 1$$

Siccome abbiamo detto che v è il primo "errore" che si incontra allora u è corretto:

$$u.d = \delta(s, u)$$

A questo punto la BFS esplora i vicini di u e quindi, se non è stato ancora visitato, anche v e succede questo:

$$v.d = u.d + 1 = \delta(s, u) + 1 = \delta(s, v)$$

In un altro caso, v non può essere gia stato scoperto perchè v può essere scoperto solo da un nodo con distanza $\delta(s, v) - 1$, quindi proprio u, nessun altro. (Corollario 4)
Dimostrazione completata.

12.5.2 DFS

La **DFS (Depth First Search)** sceglie un $v \in V$ e visita tutta la porzione del grafo raggiungibile da v seguendo percorsi il più **profondi** possibile e seguendo solo archi non ancora esplorati.
Ovviamente gli archi vengono visitati partendo dall'ultimo vertice visitato che contiene archi uscenti non visitati.
Prima di vedere il codice, stabiliamo che:

- $v.d$ = (la d sta per discovery) registra il momento in cui si incontra v per la prima volta.
- $v.F$ = (la F sta per fine visita) registra il momento in cui termina la visita di v (visitati tutti gli elementi di adj(v))
- $v.\pi$ = predecessore del nodo
- $v.status$ = può essere "A" (da visitare), "B" (Visitato), "C" (Visitato e visitati tutti gli archi uscenti).
- time = variabile globale

```
DFS(G):
    forall v in V do:
        v.status = A
        v.pi = nul
    time = 0
    forall v in V do:
        if(v.status == A):
            DFS-VISIT(G,v)

DFS-VISIT(G,u):
    time++
    u.d = time
    u.status = B
    forall v in adj(u) do:
        if(v.status == A):
            v.pi = u
        DFS-VISIT(G,v)
```

```
u.status = C
time++
u.f = time
```

Spiegazione del codice: si svolge la visita DFS per ogni sorgente, dato che un grafo può avere componenti connesse distinte.

Complessita: $\sum_{u \in V}^{\delta(u)} = |E| = m$.
Ogni sottografo generato dalla DFS è detta **Foresta DFS**.

Teorema delle parentesi: se $u, v \in V$ e $I_u = [u.d, u.f]$ e $I_v = [v.d, v.f]$ allora sono una di queste è vera:

- $I_u \cap I_v = \emptyset$ (i nodi sono distinti e nessuno è discendente dell'altro.) (un discendente di un nodo fa parte dei suoi adiacenti)
- $I_u \subset I_v$ (u è discendente di v)
- $I_v \subset I_u$ (v è discendente di u)

Classificazione degli archi su grafi orientati:

- **Archi d'albero:** $(u, v) \in E$ è un arco d'albero se $u = v.\pi$ (v è A quando si ispeziona)
- **Archi all'indietro:** se v è antenato di u (v è B quando si ispeziona)
- **Archi in avanti:** archi che non sono d'albero e v è discendente di u (v è C quando viene ispezionato)
- **Archi trasversali:** se u e v non sono discendenti uno dell'altro.
-

Teorema del cammino bianco: v è discendente di u se e solo se, al momento in cui u viene scoperto (u.d), esiste un cammino di soli nodi bianchi da u a v.

Dimostrazione:

- $u == v \implies$ il cammino è formato solo da u che è bianco al momento.
- $u \neq u \implies$ se v è discendente di u allora il suo intervallo è sottoinsieme dell'intervallo di u. Siccome v viene scoperto dopo u, è bianco all'istante u.d (v.d > u.d).

Classificazione degli archi su grafi non orientati In questo caso gli archi possono essere solo d'albero o all'indietro.
Ogni arco viene ispezionato due volte e la classificazione si fa la prima volta che si analizza.

Dimostrazione:

- Caso 1: (u, v) esplorato la prima volta da u verso v, in questo caso v è A e l'arco è d'albero.
- Caso 2: (u, v) esplorato la prima volta da v verso u, in tal caso è all'indietro.

Teorema: un grafo è ciclico se e solo se contiene un arco all'indietro.

Dimostrazione: la definizione di un arco (u, v) all'indietro dice che u è antenato di v e quindi da u si può raggiungere v. Se uniamo il cammino da u a v con l'arco all'indietro (v, u) allora si forma un ciclo.
Inoltre, gli archi di un ciclo non possono essere tutti d'albero, ce ne deve essere almeno uno all'indietro.
Se il grafo invece è orientato supponiamo che v sia il primo vertice di un ciclo ad essere scoperto e diventare grigio, gli altri vertici sul ciclo al momento sono bianchi.
Sia u il vertice che precede v, tutti i vertici sul cammino da v a u sono bianchi e per il teorema del cammino bianco u è discendente di v e quindi v è antenato di u e (u, v) è un arco all'indietro.

12.5.3 Ordinamento topologico e DAG

Un **DAG** è un **Directed Acyclic Graph**, ovvero un grafo direzionato aciclico.
Un algoritmo per un ordinamento topologico prende in input un DAG e ritorna appunto un ordinamento topologico, ovvero una sequenza di nodi di G tale che:

$$\forall (u, v) \in E, u \text{ precede } v$$

L'ordinamento topologico di un DAG non è unico.
Può essere applicato per stabilire un ordine di eventi.

```
OT(G):
    DFS(G) //ma aggiungo in testa a L u quando faccio u.f = k
    return L
```

Ha complessita quanto una DFS più gli inserimenti in L:

$$\Theta(n + m) + \Theta(n) = \Theta(n + m)$$

Teorema di correttezza dell'OT: l'algoritmo di OT(G) produce un ordinamento topologico del DAG G.

Lemma 1: G è un DAG $\Leftrightarrow$ DFS(G) non ha archi all'indietro. (e quindi non ci sono cicli, già dimostrato secondo un teorema sopra)

13 Calcolabilità

La **calcolabilità** si occupa delle questioni fondamentali circa la potenza e le limitazioni dei sistemi di calcolo.
Nacque nella prima metà del ventesimo secolo quando si scoprì l'esistenza dei problemi **non decidibili**, ovvero senza algoritmo risolutivo.
I problemi decidibili si suddividono in:

- Trattabili (costo polinomiale)
- Intrattabili (costo esponenziale)

La calcolabilità classifica gli algoritmi in decidibili e non.

13.1 Esistenza dei problemi indecidibili

Due insiemi A e B hanno lo stesson numero di elementi se e solo se esiste una biiezione tra di loro.
Inoltre, cerchiamo di enumerare ogni possibile sequenza costruita su un alfabeto finito.
Si stabilisce un ordine tra i caratteri dell'alfabeto e, a pari lunghezza, in ordine alfabetico, in tal modo si da un ordine a tutto e sapremo quanti passi ci vorranno per raggiungere tale stato.
Ad una sequenza corrisponde quindi il suo numero di passi per raggiungerla.
Non è possibile enumerare le sequenze di lunghezza infinita.
Un **problema computazionale** può essere visto come una funzione matematica che associa ad ogni insieme di dati, espressi da k numeri interi, il corrispondente risultato, indicato da j.

$$f : N^k \to N^j$$

Qualsiasi sia l'enumerazione di tali funzioni, questo insieme non è numerabile.

L'insieme degli algoritmi è numerabile e:

$$Algoritmi << Problemi$$

In quanto esistono problemi non risolvibili tramite algoritmo.
Un algoritmo dipende dal linguaggio utilizzato e consiste da una sequenza finita di operazioni.
Per quanto infiniti possano essere gli algoritmi, essi sono numerabili.

13.2 Il problema dell'arresto

Non è facile trovare un algoritmo non calcolabile.
Il problema dell'arresto è in forma decisionale:

$$Arresto : \{Istanze\} \to \{0, 1\}$$

Preso un algoritmo A e dati di input D, decidere in tempo finito se l'esecuzione di A su D termina o no, quindi con A in input:

$$Arresto(A, D) \rightarrow \{0, 1\}$$

Turing, ideatore di tale problema, riuscì a dimostrare che se un programma arbitrario si arresta e termina la sua esecuzione è una cosa impossibile e quindi è indecidibile.
Questo è dimostrabile dal fatto che, se A non terminasse allora ARRESTO non ritornerebbe mai tale informazione.
Quindi non può esistere un algoritmo che decida in tempo finito se un algoritmo arbitrario termina la sua esecuzione.
Ci sono moltissimi altri esempi:

- L'algoritmo per trovare l'equivalenza tra due programmi è indecidibile (se per ogni possibile input, producono lo stesso output)
- L'algoritmo che determina se un programma è un malware è indecidibile (ovvero se compie azioni malevoli)
- Il decimo problema di Hilbert è indecidibile (determinare se una soluzione polinomiale con coefficienti interi ha soluzioni intere (equazioni diofantee))

Algoritmo di GOLDBACH: restituisce il primo numero intero pari che non è risultato di una somma di due numeri primi qualsiasi.

13.3 Tesi di Church-Turing

Ogni modello di calcolo risolve esattamente la stessa classe di problemi, dunque si equivalgono nella possibilità di risolverli, pur avendo diversa efficienza.
Eventuali migliorie hardware o software servono solo a migliorare l'efficienza.

13.4 Problemi trattabili e intrattabili

Il problema della torre di Hanoi è intrattabile (cerca su internet).
Un altro algoritmo non trattabile è quello che calcola ogni possibile permutazione$G$ contiene una **clique** (cricca) di almeno k nodi.
Un algoritmo per risolverlo è quello che considera tutti i sottinsiemi di vertici, in ordine decrescente di cardinalità, e verifica se formano una cricca di ordine k.
Tale algoritmo non è ancora stato trovato e si sospetta che non esista.

Cammino hamiltoniano: cammino semplice che passa da tutti i vertici di G una volta sola.

Ciclo hamiltoniano: cammino hamiltoniano chiuso. Date queste due definizioni, un algoritmo per trovare un cammino hamiltoniano in un grafo non è noto, dovrebbe considerare tutte le permutazioni di vertici e verificare se sono consecutivi (a due a due adiacenti).

14 Complessità

14.1 Differenza di potenza tra calcolatori

Alcuni calcolatori sono più veloci di altri, per esempio se C_2 è k volte più veloce di C_1, allora usare C_2 per un tempo t equivale ad usare C_1 per un tempo $k \cdot t$.

14.2 Categorizzazione dei problemi

La categorizzazione dei problemi è in base alla domanda che ci facciamo:

- **Problema decisionale:** ci si chiede se vale o meno una certa proprietà. (isAbr)
- **Problema di ricerca:** ricercare un qualcosa o restituire un oggetto dato un input. (ritorno di un cammino da un grafo)
- **Problema di ottimizzazione:** si vuole trovare la migliore soluzione tra tutte le soluzioni possibili. (Algoritmi Greedy)

Molta della teoria della complessita in questa sezione è definita principalmente in termini dei problemi di decisione.
Questo perchè non ci si deve preoccupare del tempo di formulazione dell'output, in quanto è soltanto un boolean. Inoltre, la complessita di ogni problema è riconducibile alla complessita del suo relativo problema decisionale.
Per esempio, un problema di ottimizzazione può essere definito come problema decisionale:

- Problema di ottimizzazione: voglio trovare la clique più grande in un grafo G
- Problema decisionale associato: esiste una clique in G con almeno k vertici?

Il problema decisionale non è più difficile del problema di ottimizzazione: se sappiamo risolvere il problema di ottimizzazione, confrontiamo il risultato con k e quindi risolveremmo il problema decisionale di conseguenza.
Si può dire quindi che la complessita del problema di ottimizzazione ha un limite inferiore pari alla complessita del problema decisionale annesso.

14.3 Time e Space

Time(f(n)): insiemi dei problemi decisionali che possono essere risolti in tempo O(f(n))

Space(f(n)): insiemi dei problemi decisionali che possono essere risolti in spazio O(f(n))

14.4 Classi di problemi

- **classe P** è la classe dei problemi risolvibili in tempo polinomiale nella dimensione n dell'istanza di ingresso.
- **classe PSPACE:** è la classe dei problemi risolvibili in spazio polinomiale nella dimensione n dell'istanza di ingresso.
- **Classe EXP Time:** è la classe dei problemi risolvibili in tempo esponenziale nella dimensione n dell'istanza di ingresso.

Inoltre:

$$P \subseteq PSPACE$$

$$PSPACE \subseteq ExpTime \implies P \subseteq ExpTime$$

Problemi in tempo esponenziale:

- Commesso Viaggiatore
- Clique
- Cammino hamiltoniano
- Soddisfacibilità di formule booleane (SAT)
- Torre di Hanoi

14.4.1 SAT

Insieme V di variabili booleane:

- Letterale: variabile o sua negazione
- Clausola: disgiunzione (OR) di letterali

Un espressione booleana su V si dice in **forma normale congiuntiva (FNC)** se è espressa come **congiunzione di clausole** (AND di OR letterali)
Per i SAT si considerano tutti i 2^n assegnamenti possibili (possibili combinazioni) su n variabili in input e si controlla se risulta vero il risultato o meno. Questo algoritmo non è noto in quanto non si sa se è risolvibile in tempo polinomiale e quindi si presume sia esponenziale (Tutti gli SAT).

14.5 Certificati

In un problema decisionale siamo interessati a verificare se un istanza del problema soddisfa una certa proprietà, ovvero saper rispondere ad una domanda per una qualsiasi istanza in input.
Per alcuni problemi, per le **istanze accettabili** (Un input specifico accettabile) è possibile fornire un **certificato**, ovvero un **attestato breve di esistenza** di una soluzione ad un dato problema.
Un certificato di non esistenza è facile da costruire ma sicuramente non breve.
Vediamo un esempio di applicazione dove si applica il costo del certificato per determinare il costo del problema generale:
PI è un problema in tempo polinomiale se:

- Ogni istanza accettabile nel problema (ogni possibile input) di lunghezza n ammette un certificato y di lunghezza polinomiale in n
- Esiste un algoritmo di verifica polinomiale in n e applicabile a ogni coppia <x,y> che permette di attestare che l'input è accettabile

La **classe NP** è la classe dei problemi decisionali verificabili in tempo polinomiale (NON polinomiali). Questa classe è l'insieme dei problemi risolvibili in tempo polinomiale non deterministico.

$$P \subseteq NP$$

Ogni problema P ammette un certificato verificabile in tempo polinomiale, basta seguire l'algoritmo che risolve il problema per costruire il certificato.
In realtà non si sa se:

$$P = NP$$

o se:

$$P \neq NP$$

Si è certi che siano diversi, ma nessuno sa dimostrarlo.
Ma è possibile individuare i problemi più difficili all'interno della classe NP, ovvero quelli che sono in NP / P se P fosse diverso da NP, sono i problemi **NP-Completi**.
Tali problemi sono quelli provatamente più difficili della classe NP
Se esistesse un algoritmo polinomiale in grado di risolvere uno di questi problemi allora tutti tali problemi lo sarebbero e quindi:

$$P = NP$$

Se invece si potesse dimostrare che almeno uno di questi problemi non offre un algoritmo risolutivo polinomiale allora:

$$P \neq NP$$

In realtà, o li risolvo tutti in tempo polinomiale, o non ne risolvo nessuno.

14.5.1 Riduzione polinomiale

Presi due problemi A e B decisionali e I_1 e I_2 insiemi delle istanze dei due problemi, A si riduce in tempo polinomiale a B:

$$A \leq_p B$$

Se esiste una funzione $f : I_1 \rightarrow I_2$ calcolabile in tempo polinomiale tale che per ogni istanza x di A:

- x è istanza accettabile di A SE E SOLO SE f(x) lo è di B

Questo implica che se esiste un algoritmo polinomiale per risolvere B, allora esiste anche per risolvere A, ci si può ricondurre ad un problema più piccolo(?) e risolvere quello, quindi l'input viene prima tramutato in input di un problema secondario e poi viene risolto il problema secondario, questo equivale a risolvere il primo problema.
Un problema decisionale C si dice NP-completo se:

- $CinNP$
- $\forall C^{|} \in NP, C^{|} \leq_p C$

Un problema C si dice **NP-Arduo** se:

$$\forall C^{|} \in NP, C^{|} \leq_p C$$

Quindi C non è necessariamente in NP.

14.5.2 Teorema di Cook (1971)

Ha dimostrato che ogni SAT è NP-Completo.
Un problema A decisionale si dice NP-Completo se:

- $A \in NP$
- $SAT \leq_p A$

Per esempio, Clique è completo: per ogni clausola di SAT si tramutano i propri elementi in vertici:
SAT = (a OR b) AND (c OR d) AND !a
Vertici = (a,b,c,d,!a)
Gli archi invece si connette ogni elemento di ogni clausola con elementi che:

- Non sono della stessa clausola
- Sono diversi dal negativo del vertice a cui collegarsi (per esempio a non verrà collegato con !a)

Archi = ((a,c),(a,d),(b,c)(b,d)(b,!a)(c,!a)(d,!a))
In tale grafo esiste una cricca se SAT è soddisfacibile, infatti ogni cricca contiene elementi tutti di clausole diverse e la loro combinazione risolve la formula (ritornando vero per SAT).
Questo vale anche al contrario, se SAT è soddisfacibile allora esiste una clique in G.

I problemi **NP equivalenti** sono problemi che tra di loro sono appunto equivalenti...
Per esempio, SAT e clique sono tra di loro equivalenti, ma anche tutti gli NP-Completi lo sono tra di loro.

Altri prolemi importanti:

- Vertex cover per grafi
- Graph Coloring (colorare con più k colori possibili tale che due vertici adiacenti non abbiano mai lo stesso colore), questo è per esempio connesso alla colorazione della cartina politica (divisione degli stati) del mondo.

15 Altri algoritmi

15.1 Array non ordinato

15.1.1 Ricerca

Scansione lineare sequenziale fino a trovare l'elemento.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.1.2 Inserimento

Inserimento in ultima posizione (se c'è spazio disponibile).

- Caso migliore: $O(1)$
- Caso medio: $O(1)$
- Caso peggiore: $O(1)$ (oppure $O(n)$ se serve ridimensionamento)

15.1.3 Cancellazione

Ricerca dell'elemento e sostituzione con l'ultimo elemento.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.2 Array ordinato

15.2.1 Ricerca

Ricerca binaria.

- Caso migliore: $O(1)$
- Caso medio: $O(\log n)$
- Caso peggiore: $O(\log n)$

15.2.2 Inserimento

Ricerca binaria per trovare la posizione + shift degli elementi successivi.

- Caso migliore: $O(n)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.2.3 Cancellazione

Ricerca binaria + shift degli elementi successivi.

- Caso migliore: $O(n)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.3 Lista non ordinata (semplicemente concatenata)

15.3.1 Ricerca

Scansione lineare nodo per nodo.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.3.2 Inserimento

Inserimento in testa.

- Caso migliore: $O(1)$
- Caso medio: $O(1)$
- Caso peggiore: $O(1)$

15.3.3 Cancellazione

Ricerca del nodo precedente + aggiornamento dei puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.4 Lista ordinata (semplicemente concatenata)

15.4.1 Ricerca

Scansione lineare fino al valore o superamento.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.4.2 Inserimento

Ricerca posizione corretta + aggiornamento puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.4.3 Cancellazione

Ricerca nodo + aggiornamento puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.5 Lista doppia non ordinata

15.5.1 Ricerca

Scansione lineare (in avanti o bidirezionale).

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.5.2 Inserimento

Inserimento in testa (aggiornamento di due puntatori).

- Caso migliore: $O(1)$
- Caso medio: $O(1)$
- Caso peggiore: $O(1)$

15.5.3 Cancellazione

Ricerca nodo + aggiornamento di entrambi i puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.6 Lista doppia ordinata

15.6.1 Ricerca

Scansione lineare fino al valore o superamento.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.6.2 Inserimento

Ricerca posizione corretta + aggiornamento dei due puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.6.3 Cancellazione

Ricerca nodo + aggiornamento dei due puntatori.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.7 Albero binario (generico)

15.7.1 Ricerca

Visita completa dell'albero (DFS o BFS) senza proprietà di ordinamento.

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.7.2 Inserimento

Inserimento nel primo posto disponibile (ad esempio per livelli).

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.7.3 Cancellazione

Ricerca del nodo + sostituzione con l'ultimo nodo (in visita per livelli).

- Caso migliore: $O(1)$
- Caso medio: $O(n)$
- Caso peggiore: $O(n)$

15.8 Albero binario di ricerca (BST)

15.8.1 Ricerca

Confronto ricorsivo (o iterativo) partendo dalla radice: sinistra se minore, destra se maggiore.

- Caso migliore: $O(1)$
- Caso medio: $O(\log n)$ (bilanciato)
- Caso peggiore: $O(n)$ (degenere)

15.8.2 Inserimento

Ricerca della posizione corretta e inserimento come foglia.

- Caso migliore: $O(1)$
- Caso medio: $O(\log n)$ (bilanciato)
- Caso peggiore: $O(n)$ (degenere)

15.8.3 Cancellazione

Gestione dei tre casi (foglia, un figlio, due figli con predecessore/successore).

- Caso migliore: $O(1)$
- Caso medio: $O(\log n)$ (bilanciato)
- Caso peggiore: $O(n)$ (degenere)

15.9 Tabella riassuntiva

Struttura	Ricerca	Inserimento	Cancellazione
Array non ordinato	$O(n)$	$O(1)$	$O(n)$
Array ordinato	$O(\log n)$	$O(n)$	$O(n)$
Lista non ordinata	$O(n)$	$O(1)$	$O(n)$
Lista ordinata	$O(n)$	$O(n)$	$O(n)$
Lista doppia non ordinata	$O(n)$	$O(1)$	$O(n)$
Lista doppia ordinata	$O(n)$	$O(n)$	$O(n)$
Albero binario	$O(n)$	$O(n)$	$O(n)$
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$

16 Cheat Sheet (MiniMAO e MAO)

V	::=	`n` | `b`	(valori: interi o booleani)
Id	::=	sequenza di lettere, cifre, _	(identificatori)
T	::=	`int` | `bool`	(tipi base)
E	::=	V	(espressioni: valore)
	|	Id	(espressioni: variabile)
	|	E *bop* E	(espressioni: operazione binaria)
	|	*uop* E	(espressioni: operazione unaria)
	|	(E)	(espressioni: raggruppamento)
C	::=	`skip;`	(comandi: comando nullo)
	|	T Id = E;	(comandi: dichiarazione con inizializzazione)
	|	Id := E;	(comandi: assegnamento)
	|	C C	(comandi: sequenza)
	|	`if` (E) {C} `else` {C}	(comandi: condizionale)
	|	`while` (E) {C}	(comandi: ciclo while)
	|	{C}	(comandi: blocco)

Figura 1: Grammatica di MiniMao

$$\frac{}{\Gamma \vdash \mathtt{n} : \mathtt{int}}\ \text{T-Int} \qquad \frac{}{\Gamma \vdash \mathtt{b} : \mathtt{bool}}\ \text{T-Bool} \qquad \frac{\Gamma(\mathsf{Id}) = \mathsf{T}}{\Gamma \vdash \mathsf{Id} : \mathsf{T}}\ \text{T-Var}$$

$$\frac{\Gamma \vdash \mathsf{E}_1 : \mathsf{T}_1 \qquad \Gamma \vdash \mathsf{E}_2 : \mathsf{T}_2 \qquad \mathit{bop} : \mathsf{T}_1 \times \mathsf{T}_2 \to \mathsf{T}_3}{\Gamma \vdash \mathsf{E}_1\ \mathit{bop}\ \mathsf{E}_2 : \mathsf{T}_3}\ \text{T-Bop}$$

$$\frac{\Gamma \vdash \mathsf{E} : \mathsf{T} \qquad \mathit{uop} : \mathsf{T} \to \mathsf{T}'}{\Gamma \vdash \mathit{uop}\ \mathsf{E} : \mathsf{T}'}\ \text{T-Uop} \qquad \frac{\Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash (\mathsf{E}) : \mathsf{T}}\ \text{T-Group}$$

$$\frac{}{\Gamma \vdash \mathtt{skip;} : \varnothing}\ \text{T-Skip} \qquad \frac{\Gamma \vdash \mathsf{E} : \mathtt{bool} \qquad \Gamma \vdash \mathsf{C}_1 : \Gamma_1 \qquad \Gamma \vdash \mathsf{C}_2 : \Gamma_2}{\Gamma \vdash \mathtt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \mathtt{else}\ \{\mathsf{C}_2\} : \varnothing}\ \text{T-If}$$

$$\frac{\Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \mathsf{T}\ \mathsf{Id}\ \mathtt{=}\ \mathsf{E}\mathtt{;} : \{\mathsf{Id} : \mathsf{T}\}}\ \text{T-Decl} \qquad \frac{\Gamma \vdash \mathsf{C}_1 : \Gamma_1 \qquad \Gamma[\Gamma_1] \vdash \mathsf{C}_2 : \Gamma_2}{\Gamma \vdash \mathsf{C}_1\ \mathsf{C}_2 : \Gamma_1[\Gamma_2]}\ \text{T-Seq}$$

$$\frac{\Gamma(\mathsf{Id}) = \mathsf{T} \qquad \Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \mathsf{Id}\ \mathtt{:=}\ \mathsf{E}\mathtt{;} : \varnothing}\ \text{T-Assign} \qquad \frac{\Gamma \vdash \mathsf{C} : \Gamma'}{\Gamma \vdash \{\mathsf{C}\} : \varnothing}\ \text{T-Block}$$

$$\frac{\Gamma \vdash \mathsf{E} : \mathtt{bool} \qquad \Gamma \vdash \mathsf{C} : \Gamma}{\Gamma \vdash \mathtt{while}\ (\mathsf{E})\ \{\mathsf{C}\} : \varnothing}\ \text{T-While}$$

Figura 2: Type Checking di MiniMao

$$\frac{}{\langle \mathtt{n}, \rho, \sigma \rangle \Downarrow n}\ \text{E-Int} \qquad \frac{}{\langle \mathtt{b}, \rho, \sigma \rangle \Downarrow b}\ \text{E-Bool} \qquad \frac{\sigma(\rho(\mathsf{Id})) = v}{\langle \mathsf{Id}, \rho, \sigma \rangle \Downarrow v}\ \text{E-Var}$$

$$\frac{\langle \mathsf{E}_1, \rho, \sigma \rangle \Downarrow v_1 \qquad \langle \mathsf{E}_2, \rho, \sigma \rangle \Downarrow v_2 \qquad v = v_1\ \mathit{bop}\ v_2}{\langle \mathsf{E}_1\ \mathit{bop}\ \mathsf{E}_2, \rho, \sigma \rangle \Downarrow v}\ \text{E-Bop}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma \rangle \Downarrow v \qquad v' = \mathit{uop}\ v}{\langle \mathit{uop}\ \mathsf{E}, \rho, \sigma \rangle \Downarrow v'}\ \text{E-Uop} \qquad \frac{\langle \mathsf{E}, \rho, \sigma \rangle \Downarrow v}{\langle (\mathsf{E}), \rho, \sigma \rangle \Downarrow v}\ \text{E-Group}$$

Figura 3: Semantica delle espressioni di MiniMao

$$\frac{}{\langle \texttt{skip;}, \rho, \sigma\rangle \to (\rho, \sigma)} \text{ C-Skip}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow v \qquad l \notin \mathrm{dom}(\sigma)}{\langle \mathsf{T}\ \mathsf{Id}\ \texttt{=}\ \mathsf{E}\texttt{;}, \rho, \sigma\rangle \to (\rho[\mathsf{Id} \mapsto l], \sigma[l \mapsto v])} \text{ C-Decl}$$

$$\frac{\rho(\mathsf{Id}) = l \qquad \langle \mathsf{E}, \rho, \sigma\rangle \Downarrow v}{\langle \mathsf{Id}\ \texttt{:=}\ \mathsf{E}\texttt{;}, \rho, \sigma\rangle \to (\rho, \sigma[l \mapsto v])} \text{ C-Assign}$$

$$\frac{\langle \mathsf{C}_1, \rho, \sigma\rangle \to (\rho_1, \sigma_1) \qquad \langle \mathsf{C}_2, \rho_1, \sigma_1\rangle \to (\rho_2, \sigma_2)}{\langle \mathsf{C}_1\ \mathsf{C}_2, \rho, \sigma\rangle \to (\rho_2, \sigma_2)} \text{ C-Seq}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow \mathit{true} \qquad \langle \mathsf{C}_1, \rho, \sigma\rangle \to (\rho_1, \sigma_1)}{\langle \texttt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \texttt{else}\ \{\mathsf{C}_2\}, \rho, \sigma\rangle \to (\rho, \sigma_1)} \text{ C-If-True}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow \mathit{false} \qquad \langle \mathsf{C}_2, \rho, \sigma\rangle \to (\rho_2, \sigma_2)}{\langle \texttt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \texttt{else}\ \{\mathsf{C}_2\}, \rho, \sigma\rangle \to (\rho, \sigma_2)} \text{ C-If-False}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow \mathit{true} \quad \langle \mathsf{C}, \rho, \sigma\rangle \to (\rho_1, \sigma_1) \quad \langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma_1\rangle \to (\rho_2, \sigma_2)}{\langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma_2)} \text{ C-Wh-True}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow \mathit{false}}{\langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma)} \text{ C-Wh-False}$$

$$\frac{\langle \mathsf{C}, \rho, \sigma\rangle \to (\rho', \sigma')}{\langle \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma')} \text{ C-Block}$$

Figura 4: Semantica dei comandi di MiniMao

V	::=	n | b | d | c	(valori: interi, booleani, ...)
Id	::=	sequenza di lettere, cifre, _	(identificatori)
Ids	::=	Id | Ids, Id	(liste di identificatori)
Tb	::=	int | bool | double | char	(tipi base: interi, booleani, ...)
Tc	::=	Tb[]	(tipi composti: array)
T	::=	Tb | Tc	(tipi: base o composti)
Ds	::=	T Id | T Id, Ds	(liste di dichiarazioni)
E	::=	V	(espressioni: valore)
	|	Id	(espressioni: variabile)
	|	E *bop* E	(espressioni: operazione binaria)
	|	*uop* E	(espressioni: operazione unaria)
	|	(E)	(espressioni: raggruppamento)
	|	Id[E]	(espressioni: accesso elemento array)
	|	new Tb[E]	(espressioni: nuovo array default)
	|	$[V_1, \ldots, V_n]$	(espressioni: nuovo array esplicito)
	|	Id.length	(espressioni: lunghezza di un array)
	|	T(E)	(espressioni: cast esplicito)
	|	Id(Es)	(espressioni: invocazione di funzione)
	|	{C return E;}	(espressioni: blocco side effect)
	|	(E ? E : E)	(espressioni: condizionale)
Es	::=	E | Es, E	(liste di espressioni)
C	::=	skip;	(comandi: comando nullo)
	|	Ds = Es;	(comandi: dichiarazione multipla)
	|	Ids := Es;	(comandi: assegnamento multiplo)
	|	$Id[E_1]$:= E_2;	(comandi: assegnamento ad array)
	|	C C	(comandi: sequenza)
	|	if (E) {C} else {C}	(comandi: condizionale)
	|	while (E) {C}	(comandi: ciclo while)
	|	{C}	(comandi: blocco)
	|	Id(Ds){C}	(comandi: dichiarazione di procedura)
	|	Id(Es);	(comandi: invocazione di procedura)
	|	T Id(Ds) E	(comandi: dichiarazione di funzione)

Figura 5: Grammatica di Mao

$$\frac{}{\Gamma \vdash \texttt{n} : \texttt{int}}\ \text{T-Int} \qquad \frac{}{\Gamma \vdash \texttt{b} : \texttt{bool}}\ \text{T-Bool} \qquad \frac{}{\Gamma \vdash \texttt{d} : \texttt{double}}\ \text{T-Double}$$

$$\frac{}{\Gamma \vdash \texttt{c} : \texttt{char}}\ \text{T-Char} \qquad \frac{\Gamma(\mathsf{Id}) = \mathsf{T}}{\Gamma \vdash \mathsf{Id} : \mathsf{T}}\ \text{T-Var} \qquad \frac{\Gamma(\mathsf{Id}) = \mathsf{Tb}[\,] \qquad \Gamma \vdash \mathsf{E} : \texttt{int}}{\Gamma \vdash \mathsf{Id}[\mathsf{E}] : \mathsf{Tb}}\ \text{T-At}$$

$$\frac{\Gamma \vdash \mathsf{E}_1 : \mathsf{T}_1 \qquad \Gamma \vdash \mathsf{E}_2 : \mathsf{T}_2 \qquad \mathit{bop} : \mathsf{T}_1 \times \mathsf{T}_2 \to \mathsf{T}_3}{\Gamma \vdash \mathsf{E}_1\ \mathit{bop}\ \mathsf{E}_2 : \mathsf{T}_3}\ \text{T-Bop}$$

$$\frac{\Gamma \vdash \mathsf{E} : \mathsf{T} \qquad \mathit{uop} : \mathsf{T} \to \mathsf{T}'}{\Gamma \vdash \mathit{uop}\ \mathsf{E} : \mathsf{T}'}\ \text{T-Uop} \qquad \frac{\Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash (\mathsf{E}) : \mathsf{T}}\ \text{T-Group}$$

$$\frac{\Gamma \vdash \mathsf{E} : \texttt{int}}{\Gamma \vdash \texttt{new}\ \mathsf{Tb}[\mathsf{E}] : \mathsf{Tb}[\,]}\ \text{T-New} \qquad \frac{\Gamma \vdash \mathsf{V}_1 : \mathsf{Tb} \qquad \cdots \qquad \Gamma \vdash \mathsf{V}_n : \mathsf{Tb}}{\Gamma \vdash [\mathsf{V}_1, \ldots, \mathsf{V}_n] : \mathsf{Tb}[\,]}\ \text{T-Array}$$

$$\frac{\Gamma(\mathsf{Id}) = \mathsf{Tb}[\,]}{\Gamma \vdash \mathsf{Id}.\texttt{length} : \texttt{int}}\ \text{T-Length} \qquad \frac{\Gamma \vdash \mathsf{E} : \mathsf{T}'}{\Gamma \vdash \mathsf{T}(\mathsf{E}) : \mathsf{T}}\ \text{T-Cast}$$

$$\frac{\Gamma(\mathsf{Id}) = (\prod_{i=1}^{n} \mathsf{T}_i) \to \mathsf{T} \qquad \forall i \in [1, n].\ \Gamma \vdash \mathsf{E}_i : \mathsf{T}_i}{\Gamma \vdash \mathsf{Id}(\mathsf{E}_1, \ldots, \mathsf{E}_n) : \mathsf{T}}\ \text{T-F-Call}$$

$$\frac{\Gamma \vdash \mathsf{C} : \Gamma' \qquad \Gamma[\Gamma'] \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \{\mathsf{C}\ \texttt{return}\ \mathsf{E};\} : \mathsf{T}}\ \text{T-Return}$$

$$\frac{\Gamma \vdash \mathsf{E} : \texttt{bool} \qquad \Gamma \vdash \mathsf{E}_1 : \mathsf{T} \qquad \Gamma \vdash \mathsf{E}_2 : \mathsf{T}}{\Gamma \vdash (\mathsf{E}\ ?\ \mathsf{E}_1 : \mathsf{E}_2) : \mathsf{T}}\ \text{T-Econd}$$

Figura 6: Type Checking delle espressioni di Mao

$$\frac{}{\Gamma \vdash \texttt{skip;} : \varnothing}\ \text{T-Skip} \qquad \frac{\Gamma \vdash \mathsf{E} : \texttt{bool} \qquad \Gamma \vdash \mathsf{C}_1 : \Gamma_1 \qquad \Gamma \vdash \mathsf{C}_2 : \Gamma_2}{\Gamma \vdash \texttt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \texttt{else}\ \{\mathsf{C}_2\} : \varnothing}\ \text{T-If}$$

$$\frac{\Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \mathsf{T}\ \mathsf{Id} = \mathsf{E}; : \{\mathsf{Id} : \mathsf{T}\}}\ \text{T-Decl} \qquad \frac{\Gamma \vdash \mathsf{E}_1 : \mathsf{T}_1 \qquad \Gamma \vdash \mathsf{Ds} = \mathsf{Es}; : \Gamma'}{\Gamma \vdash \mathsf{T}_1\ \mathsf{Id}_1, \mathsf{Ds} = \mathsf{E}_1, \mathsf{Es}; : \Gamma'[\mathsf{Id}_1 : \mathsf{T}_1]}\ \text{T-Decls}$$

$$\frac{\Gamma(\mathsf{Id}) = \mathsf{T} \qquad \Gamma \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \mathsf{Id} := \mathsf{E}; : \varnothing}\ \text{T-Assign}$$

$$\frac{\Gamma(\mathsf{Id}_1) = \mathsf{T}_1 \qquad \Gamma \vdash \mathsf{E}_1 : \mathsf{T}_1 \qquad \Gamma \vdash \mathsf{Ids} := \mathsf{Es}; : \Gamma'}{\Gamma \vdash \mathsf{Id}_1, \mathsf{Ids} := \mathsf{E}_1, \mathsf{Es}; : \varnothing}\ \text{T-Assigns}$$

$$\frac{\Gamma \vdash \mathsf{C}_1 : \Gamma_1 \qquad \Gamma[\Gamma_1] \vdash \mathsf{C}_2 : \Gamma_2}{\Gamma \vdash \mathsf{C}_1\ \mathsf{C}_2 : \Gamma_1[\Gamma_2]}\ \text{T-Seq} \qquad \frac{\Gamma \vdash \mathsf{C} : \Gamma'}{\Gamma \vdash \{\mathsf{C}\} : \varnothing}\ \text{T-Block}$$

$$\frac{\Gamma \vdash \mathsf{E} : \texttt{bool} \qquad \Gamma \vdash \mathsf{C} : \Gamma}{\Gamma \vdash \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\} : \varnothing}\ \text{T-While}$$

$$\frac{\Gamma(\mathsf{Id}) = \mathsf{Tb}[\,] \qquad \Gamma \vdash \mathsf{E}_1 : \texttt{int} \qquad \Gamma \vdash \mathsf{E}_2 : \mathsf{Tb}}{\Gamma \vdash \mathsf{Id}[\mathsf{E}_1] := \mathsf{E}_2; : \varnothing}\ \text{T-Assign-Array}$$

$$\frac{\mathsf{Id} : (\prod_{i=1}^{n} \mathsf{T}_i) \to \texttt{void}, \mathsf{Id}_1 : \mathsf{T}_1, \ldots, \mathsf{Id}_n : \mathsf{T}_n \vdash \mathsf{C} : \Gamma'}{\Gamma \vdash \mathsf{Id}(\mathsf{T}_1\ \mathsf{Id}_1, \ldots, \mathsf{T}_n\ \mathsf{Id}_n)\{\mathsf{C}\} : \{\mathsf{Id} : (\prod_{i=1}^{n} \mathsf{T}_i) \to \texttt{void}\}}\ \text{T-Proc-Rec}$$

$$\frac{\Gamma(\mathsf{Id}) = (\prod_{i=1}^{n} \mathsf{T}_i) \to \texttt{void} \qquad \forall i \in [1, n].\ \Gamma \vdash \mathsf{E}_i : \mathsf{T}_i}{\Gamma \vdash \mathsf{Id}(\mathsf{E}_1, \ldots, \mathsf{E}_n); : \varnothing}\ \text{T-P-Call}$$

$$\frac{\{\mathsf{Id} : (\prod_{i=1}^{n} \mathsf{T}_i) \to \mathsf{T}, \mathsf{Id}_1 : \mathsf{T}_1, \ldots, \mathsf{Id}_n : \mathsf{T}_n\} \vdash \mathsf{E} : \mathsf{T}}{\Gamma \vdash \mathsf{T}\ \mathsf{Id}(\mathsf{T}_1\ \mathsf{Id}_1, \ldots, \mathsf{T}_n\ \mathsf{Id}_n)\ \mathsf{E} : \{\mathsf{Id} : (\prod_{i=1}^{n} \mathsf{T}_i) \to \mathsf{T}\}}\ \text{T-Func-Rec}$$

Figura 7: TypeChecking dei comandi di Mao

$$\frac{}{\langle \mathsf{v}, \rho, \sigma\rangle \Downarrow (v, \sigma)}\ \textsc{E-Val} \qquad \frac{\sigma(\rho(\mathsf{Id})) = v}{\langle \mathsf{Id}, \rho, \sigma\rangle \Downarrow (v, \sigma)}\ \textsc{E-Var}$$

$$\frac{\langle \mathsf{E}_1, \rho, \sigma\rangle \Downarrow (v_1, \sigma_1) \qquad \langle \mathsf{E}_2, \rho, \sigma_1\rangle \Downarrow (v_2, \sigma_2) \qquad v = v_1\ bop\ v_2}{\langle \mathsf{E}_1\ bop\ \mathsf{E}_2, \rho, \sigma\rangle \Downarrow (v, \sigma_2)}\ \textsc{E-Bop}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (v, \sigma') \qquad v' = uop\ v}{\langle uop\ \mathsf{E}, \rho, \sigma\rangle \Downarrow (v', \sigma')}\ \textsc{E-Uop} \qquad \frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (v, \sigma')}{\langle (\mathsf{E}), \rho, \sigma\rangle \Downarrow (v, \sigma')}\ \textsc{E-Group}$$

$$\frac{\sigma(\rho(\mathsf{Id})) = l_b \qquad \langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (i, \sigma_1) \qquad 0 \leq i < \sigma_1(l_b) \qquad \sigma_1(l_b[i]) = v}{\langle \mathsf{Id}[\mathsf{E}], \rho, \sigma\rangle \Downarrow (v, \sigma_1)}\ \textsc{E-At}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (n, \sigma_1) \qquad n > 0 \qquad l_b, l_b[0], \ldots, l_b[n-1] \notin dom(\sigma_1)}{\langle \texttt{new}\ \mathsf{Tb}[\mathsf{E}], \rho, \sigma\rangle \Downarrow (l_b, \sigma_1[l_b \mapsto n, l_b[0] \mapsto v, \ldots, l_b[n-1] \mapsto v])}\ \textsc{E-New}$$

$$\frac{n > 0 \qquad l_b, l_b[0], \ldots, l_b[n-1] \notin dom(\sigma)}{\langle [\mathsf{V}_1, \ldots, \mathsf{V}_n], \rho, \sigma\rangle \Downarrow (l_b, \sigma[l_b \mapsto n, l_b[0] \mapsto v_1, \ldots, l_b[n-1] \mapsto v_n])}\ \textsc{E-Array}$$

$$\frac{\sigma(\rho(\mathsf{Id})) = l_b \qquad \sigma(l_b) = n}{\langle \mathsf{Id}.\texttt{length}, \rho, \sigma\rangle \Downarrow (n, \sigma)}\ \textsc{E-Length}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (n, \sigma_1)}{\langle \texttt{double}(\mathsf{E}), \rho, \sigma\rangle \Downarrow (n, \sigma_1)}\ \textsc{E-Cast-I2D} \qquad \frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (d, \sigma_1)}{\langle \texttt{int}(\mathsf{E}), \rho, \sigma\rangle \Downarrow (\lfloor d \rfloor, \sigma_1)}\ \textsc{E-Cast-D2I}$$

$$\frac{\langle \mathsf{C}, \rho, \sigma\rangle \to (\rho', \sigma') \qquad \langle \mathsf{E}, \rho', \sigma'\rangle \Downarrow (v, \sigma'')}{\langle \{\mathsf{C}\ \texttt{return}\ \mathsf{E};\}, \rho, \sigma\rangle \Downarrow (v, \sigma'')}\ \textsc{E-Return}$$

$$\frac{\rho(\mathsf{Id}) = \lambda(\mathsf{Ds}).\mathsf{E} \qquad \langle \{\mathsf{Ds}\ \texttt{=}\ \mathsf{Es};\ \texttt{return}\ \mathsf{E};\}, \rho, \sigma\rangle \Downarrow (v, \sigma')}{\langle \mathsf{Id}(\mathsf{Es}), \rho, \sigma\rangle \Downarrow (v, \sigma')}\ \textsc{E-Call}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (true, \sigma') \qquad \langle \mathsf{E}_1, \rho, \sigma'\rangle \Downarrow (v_1, \sigma_1)}{\langle (\mathsf{E}\ ?\ \mathsf{E}_1 : \mathsf{E}_2), \rho, \sigma\rangle \Downarrow (v_1, \sigma_1)}\ \textsc{E-Econd-True}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (false, \sigma') \qquad \langle \mathsf{E}_2, \rho, \sigma'\rangle \Downarrow (v_2, \sigma_2)}{\langle (\mathsf{E}\ ?\ \mathsf{E}_1 : \mathsf{E}_2), \rho, \sigma\rangle \Downarrow (v_2, \sigma_2)}\ \textsc{E-Econd-False}$$

Figura 8: Semantica delle espressioni di Mao

$$\frac{}{\langle \texttt{skip;}, \rho, \sigma\rangle \to (\rho, \sigma)}\ \text{C-Skip} \qquad \frac{\langle \mathsf{C}, \rho, \sigma\rangle \to (\rho', \sigma')}{\langle \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma')}\ \text{C-Block}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (v, \sigma') \qquad l \notin \mathrm{dom}(\sigma')}{\langle \mathsf{T}\ \mathsf{Id} = \mathsf{E};, \rho, \sigma\rangle \to (\rho[\mathsf{Id} \mapsto l], \sigma'[l \mapsto v])}\ \text{C-Decl}$$

$$\frac{\langle \mathsf{E}_1, \rho, \sigma\rangle \Downarrow (v_1, \sigma_1) \qquad \langle \mathsf{Ds} = \mathsf{Es};, \rho, \sigma_1\rangle \to (\rho', \sigma') \qquad l_1 \notin dom(\sigma')}{\langle \mathsf{T}_1\ \mathsf{Id}_1, \mathsf{Ds} = \mathsf{E}_1, \mathsf{Es};, \rho, \sigma\rangle \to (\rho'[\mathsf{Id}_1 \mapsto l_1], \sigma'[l_1 \mapsto v_1])}\ \text{C-Decls}$$

$$\frac{\rho(\mathsf{Id}) = l \qquad \langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (v, \sigma')}{\langle \mathsf{Id}\ := \mathsf{E};, \rho, \sigma\rangle \to (\rho, \sigma'[l \mapsto v])}\ \text{C-Assign}$$

$$\frac{\langle \mathsf{E}_1, \rho, \sigma\rangle \Downarrow (v_1, \sigma_1) \qquad \langle \mathsf{Ids}\ := \mathsf{Es};, \rho, \sigma_1\rangle \to (\rho', \sigma') \qquad \rho'(\mathsf{Id}_1) = l_1}{\langle \mathsf{Id}_1, \mathsf{Ids}\ := \mathsf{E}_1, \mathsf{Es};, \rho, \sigma\rangle \to (\rho', \sigma'[l_1 \mapsto v_1])}\ \text{C-Assigns}$$

$$\frac{\begin{array}{c}\sigma(\rho(\mathsf{Id})) = l_b \\ \langle \mathsf{E}_1, \rho, \sigma\rangle \Downarrow (i, \sigma_1) \qquad \langle \mathsf{E}_2, \rho, \sigma_1\rangle \Downarrow (v, \sigma_2) \qquad 0 \leq i < \sigma_2(l_b)\end{array}}{\langle \mathsf{Id}[\mathsf{E}_1]\ := \mathsf{E}_2;, \rho, \sigma\rangle \to (\rho, \sigma_2[l_b[i] \mapsto v])}\ \text{C-Assign-Pos}$$

$$\frac{\langle \mathsf{C}_1, \rho, \sigma\rangle \to (\rho_1, \sigma_1) \qquad \langle \mathsf{C}_2, \rho_1, \sigma_1\rangle \to (\rho_2, \sigma_2)}{\langle \mathsf{C}_1\ \mathsf{C}_2, \rho, \sigma\rangle \to (\rho_2, \sigma_2)}\ \text{C-Seq}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (\mathit{true}, \sigma') \qquad \langle \mathsf{C}_1, \rho, \sigma'\rangle \to (\rho_1, \sigma_1)}{\langle \texttt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \texttt{else}\ \{\mathsf{C}_2\}, \rho, \sigma\rangle \to (\rho, \sigma_1)}\ \text{C-If-True}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (\mathit{false}, \sigma') \qquad \langle \mathsf{C}_2, \rho, \sigma'\rangle \to (\rho_2, \sigma_2)}{\langle \texttt{if}\ (\mathsf{E})\ \{\mathsf{C}_1\}\ \texttt{else}\ \{\mathsf{C}_2\}, \rho, \sigma\rangle \to (\rho, \sigma_2)}\ \text{C-If-False}$$

$$\frac{\begin{array}{c}\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (\mathit{true}, \sigma') \\ \langle \mathsf{C}, \rho, \sigma'\rangle \to (\rho_1, \sigma_1) \qquad \langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma_1\rangle \to (\rho_2, \sigma_2)\end{array}}{\langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma_2)}\ \text{C-Wh-True}$$

$$\frac{\langle \mathsf{E}, \rho, \sigma\rangle \Downarrow (\mathit{false}, \sigma')}{\langle \texttt{while}\ (\mathsf{E})\ \{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho, \sigma')}\ \text{C-Wh-False}$$

$$\frac{\rho' = \rho[\mathsf{Id} \mapsto \lambda(\mathsf{Ds}).\{\mathsf{C}\}]}{\langle \mathsf{Id}(\mathsf{Ds})\{\mathsf{C}\}, \rho, \sigma\rangle \to (\rho', \sigma)}\ \text{C-Proc} \qquad \frac{\rho' = \rho[\mathsf{Id} \mapsto \lambda(\mathsf{Ds}).\mathsf{E}]}{\langle \mathsf{T}\ \mathsf{Id}(\mathsf{Ds})\ \mathsf{E}, \rho, \sigma\rangle \to (\rho', \sigma)}\ \text{C-Func}$$

$$\frac{\rho(\mathsf{Id}) = \lambda(\mathsf{Ds}).\{\mathsf{C}\} \qquad \langle \{\mathsf{Ds} = \mathsf{Es};\ \mathsf{C}\}, \rho, \sigma\rangle \to (\rho', \sigma')}{\langle \mathsf{Id}(\mathsf{Es}), \rho, \sigma\rangle \to (\rho', \sigma')}\ \text{C-Call}$$

Figura 9: Semantica dei comandi di Mao

17 Altro

La condizione da controllare nei comandi condizionali si chiama **guardia**.